Homework Assignments

- HW1 will be released tonight (due Sep 28th).
- HW1 is *not team-based* but others will be.
- Teams of 2-3 students are preferred for HW2-4.
- Solo-team requests are rarely approved.
- HW1 should be an easy start. Use the time to assemble your team / conceive your project.
- You have 3 late days across all assignments.

Course Project

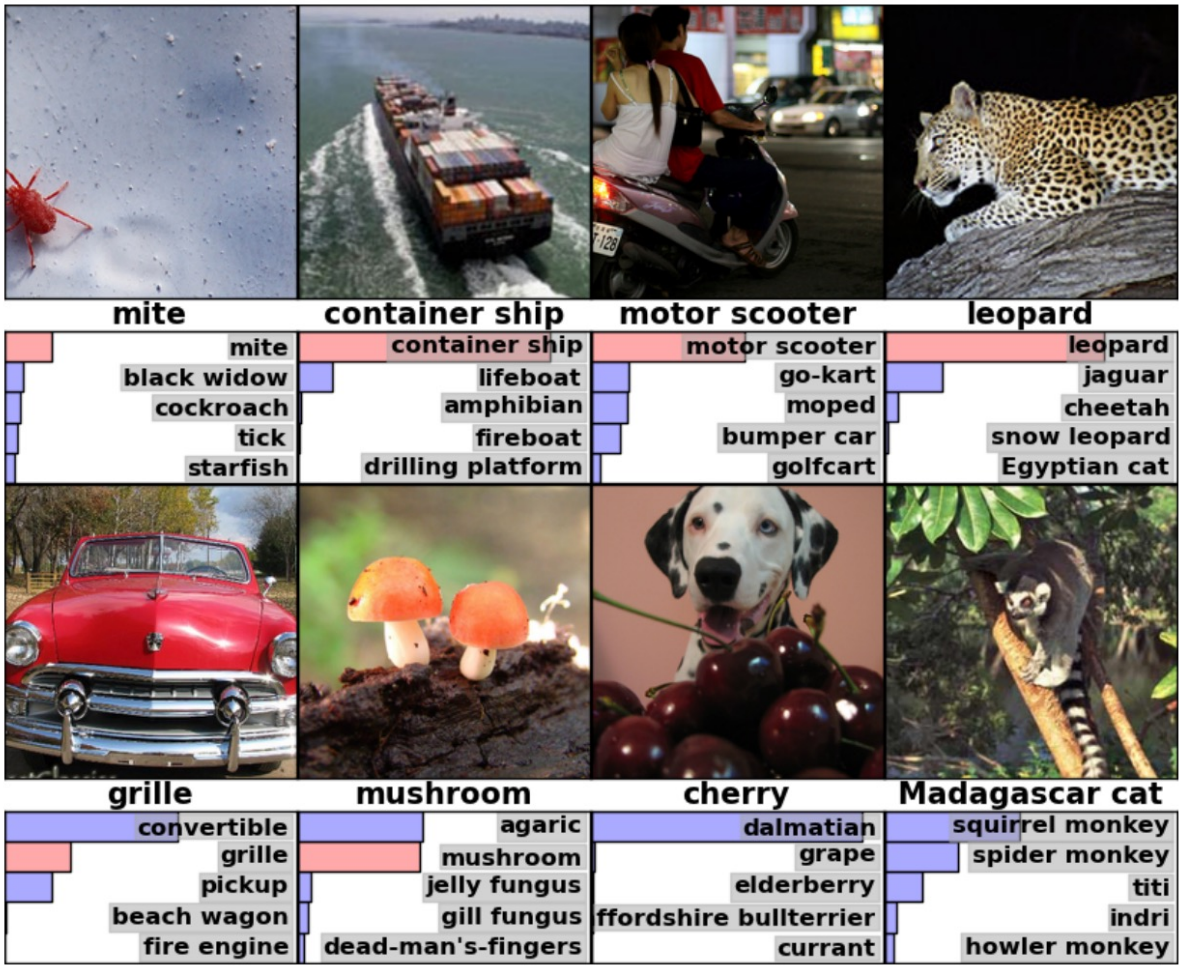
- Team-based (preferred team size: 3 students)
- Anything related to computer vision
  - A study / benchmark of existing vision methods
  - Extending an existing vision algorithm
  - Design a new vision algorithm
  - Adapting a vision method for a new application
  - …
- Must present non-trivial conceptual innovation (e.g., beyond running code from GitHub)
- Three key milestones: proposal -> mid-term report -> final presentation

Let us talk about the use of AI …

Introduction to Deep Neural Networks

Many Slides from Marc'Aurelio Ranzato and Abhinav Gupta

Image Classification



"ImageNet Classification with Deep Convolutional Neural Networks",
Krizhevsky, Sutskever, Hinton, *NIPS,* 2012

Object Detection and Segmentation

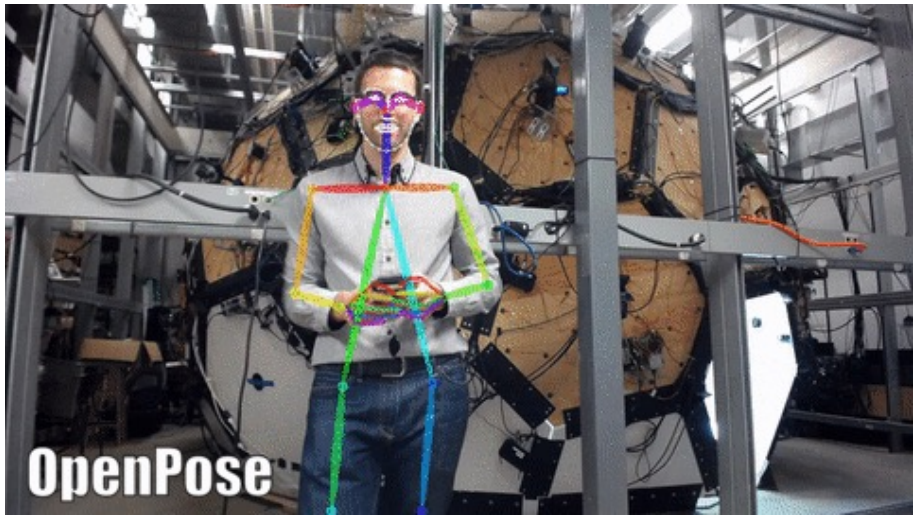


“Mask RCNN”, He, Gkioxari, Dollár, Girshick, *ICCV,* 2017

Pose Estimation



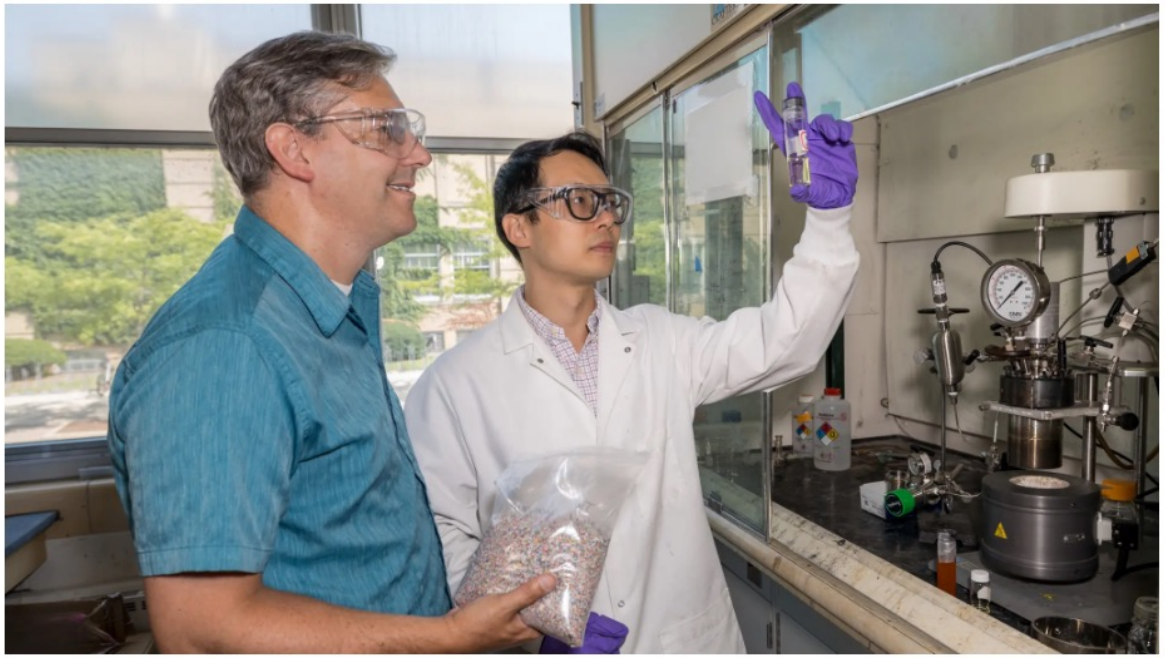




“OpenPose: realtime multi-person 2D pose estimation using Part Affinity Fields”,
Cao, Hidalgo, Simon, Wei and Sheikh, ArXiv 2018

Visual Question Answering

You



You

What is the color of the gloves worn by the person on the left?

ChatGPT

The individual on the left is not wearing gloves; their hands are bare. It's the person on the right who is wearing purple gloves.

Recognition as Supervised Learning

Image Classification





"dog"

Classification
categorical labels

Pose Estimation

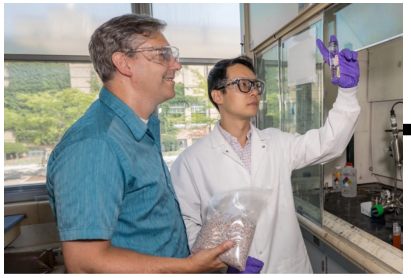






Regression
continuous values

VQA / Detection





"The person …"

Structured prediction
list of elements

Supervised Learning

$\{x_i, y_i\}$ $\mathrm{i} = 1$ ... N training dataset

x_i	*i*-th input training example
y_i	*i*-th target output
N	number of training examples



Goal: predict the target ouput of unseen inputs

* Assuming training samples and new inputs are IID

Supervised Learning for Visual Recognition



Ideal Feature Extractor → Ideal Decision Function →

window, top-left
clock, top-middle
shelf, left
…

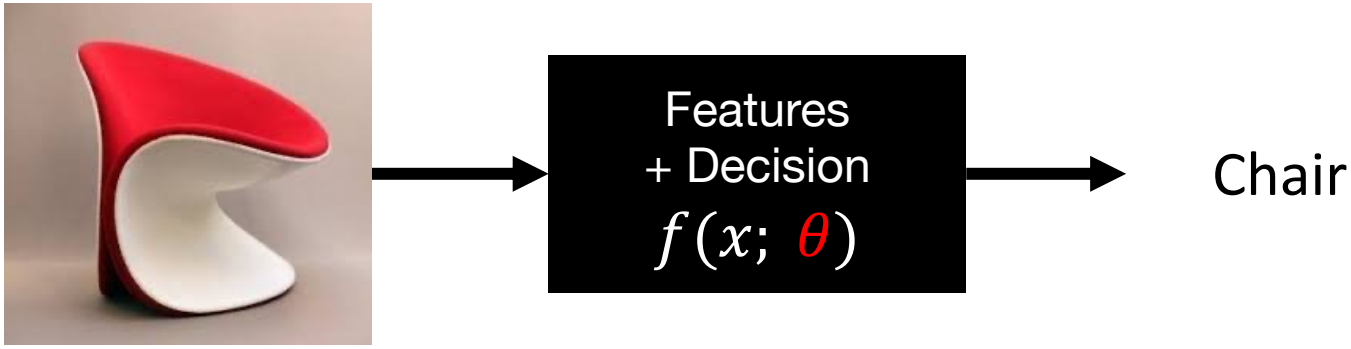
drawing, middle
hat, bottom right

Q: What types of features shall we consider?

Q: What types of decision functions shall we consider?

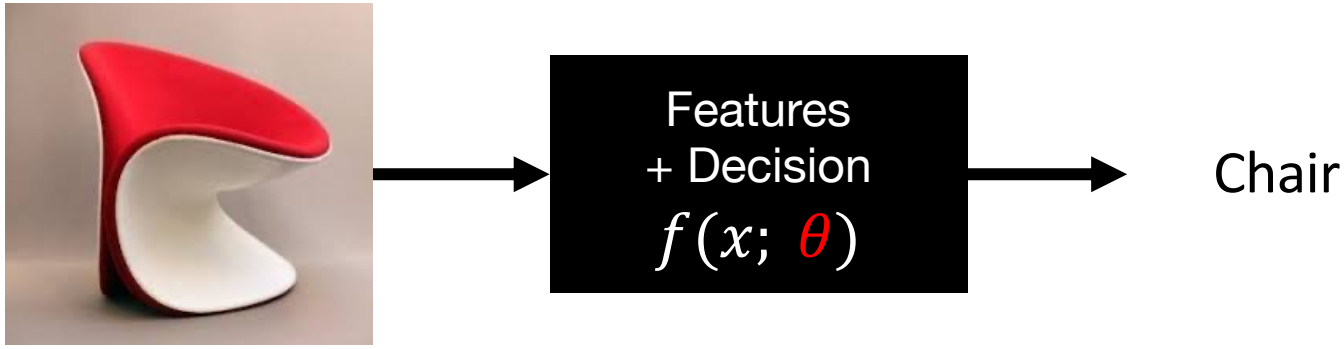
Deep Learning for Visual Recognition

- What if we only learn a single function?
  - Directly mapping from the data to the label
  - Combining features & decision function
  - Defined by its parameters θ
- What type of functions shall we consider for *f?*



Deep Learning for Visual Recognition

- What type of functions shall we consider for *f?*



Proposal: Composing a set of simple (nonlinear) functions *g*

$$f(\boldsymbol{x};\boldsymbol{\theta}) = g_1(\dots g_{n-1}(g_n(\boldsymbol{x};\boldsymbol{\theta_n}),\boldsymbol{\theta_{n-1}})\dots,\boldsymbol{\theta_1})$$

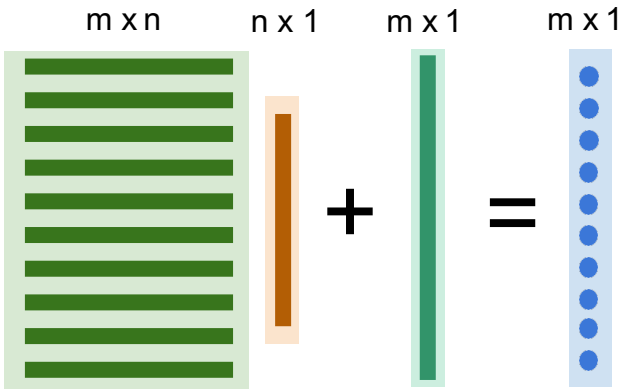
Example: $\mathbf{a} = \max(0, \boldsymbol{W^T x} + \boldsymbol{b}) = g(\boldsymbol{x};\boldsymbol{W},\boldsymbol{b})$

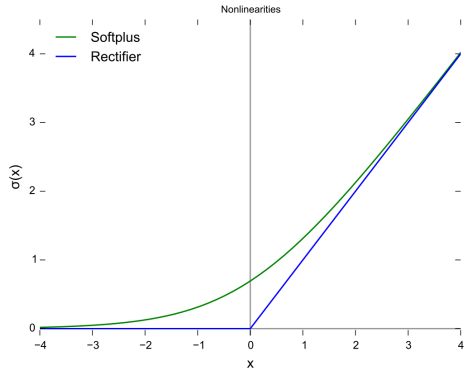
Deep Learning

- (Deep) Neural Network:
  Composing a set of simple (nonlinear) functions *g*

$$f(\boldsymbol{x};\boldsymbol{\theta}) = g_1(\dots g_{n-1}(g_n(\boldsymbol{x};\boldsymbol{\theta_n}),\boldsymbol{\theta_{n-1}})\dots,\boldsymbol{\theta_1})$$

- Key Elements:
  Linear/Multilinear operations + Nonlinear activations





Deep Learning

Deep Learning: Composing a set of simple nonlinear functions g

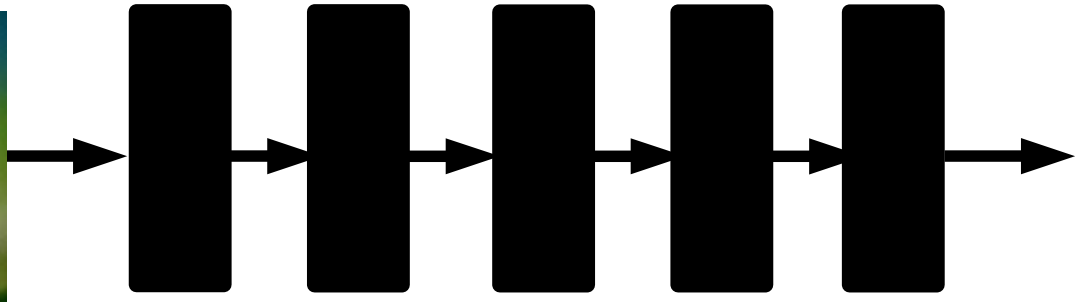
$$f(\boldsymbol{x};\boldsymbol{\theta}) = g_1(\dots g_{n-1}(g_n(\boldsymbol{x};\boldsymbol{\theta_n}),\boldsymbol{\theta_{n-1}})\dots,\boldsymbol{\theta_1})$$

- Make predictions by using a sequence of non-linear processing stages
- The resulting intermediate representations can be interpreted as a feature hierarchy
- The whole system is jointly learned from data
- The same architecture can support various tasks

Supervised Deep Learning

Image Classification

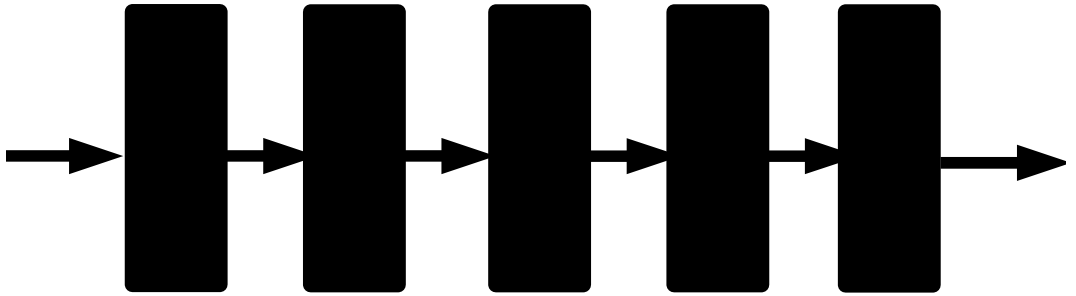




“dog”

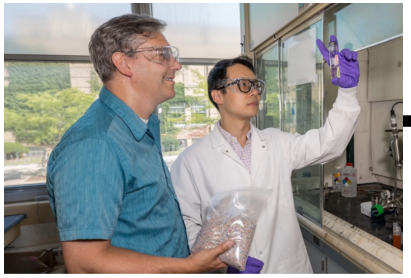
Pose Estimation

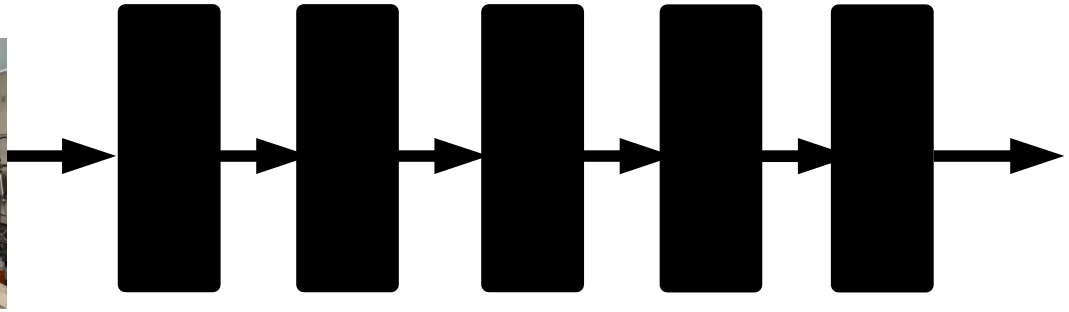






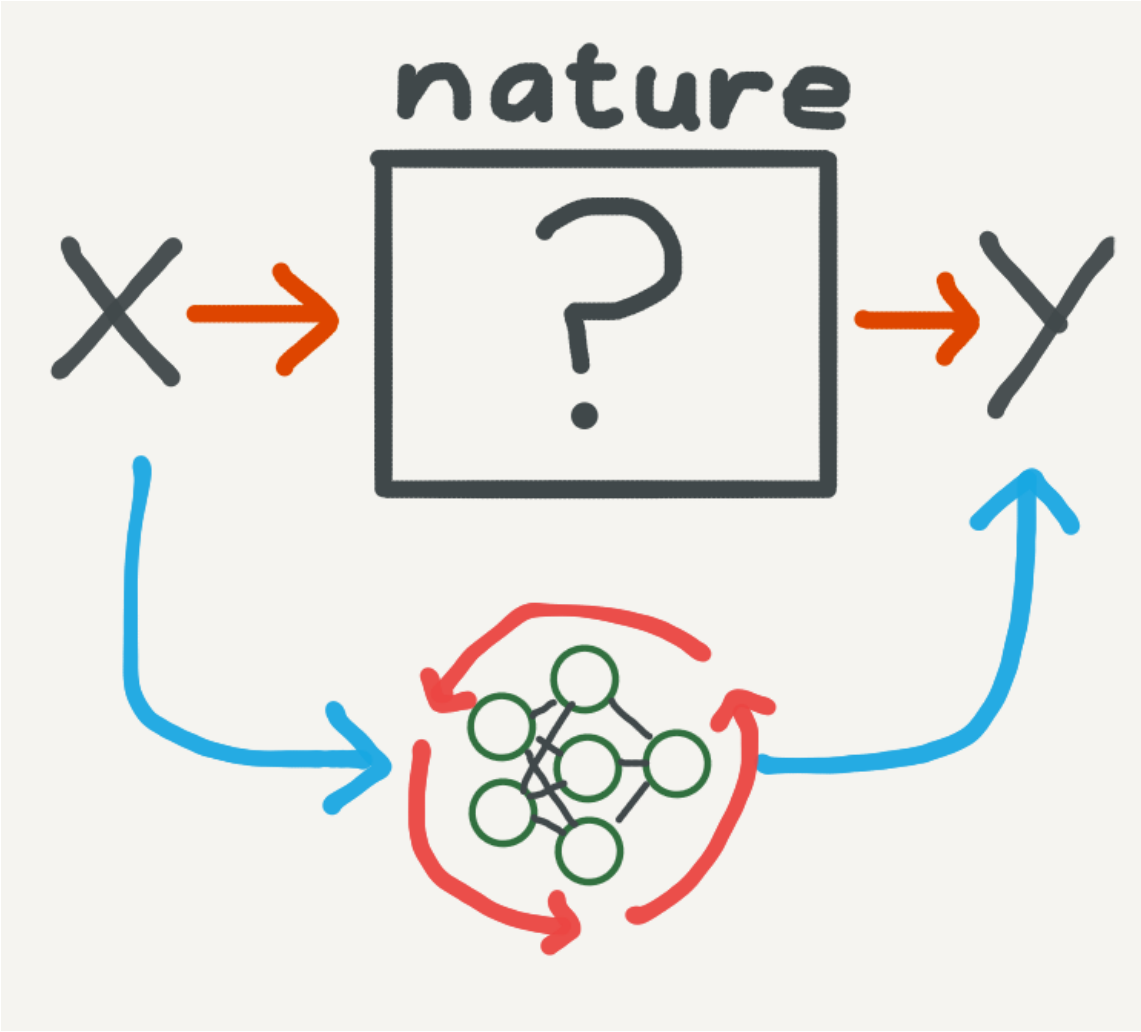
VQA / Detection





“The Person…”

Supervised Deep Learning



"**Statistical Modeling: The Two Cultures**", Brieman, Statist. Sci., 2001

A Gallery of Deep Learning Models

How to design individual nonlinear functions?

- Fully connected + activation
- Convolution + activation
- Recurrent unit
- Self-attention
- Spiking neural unit
- …

A Gallery of Deep Learning Models

How to design individual nonlinear functions?

- Fully connected + activation
- Convolution + activation
- Recurrent unit
- Self-attention
- Spiking neural unit
- …

How to compose these functions?

- Fully connected neural networks
- Convolutional neural networks
- Recurrent neural networks
- Transformer networks
- Spiking neural networks
- …

A Gallery of Deep Learning Models

How to design individual nonlinear functions?



How to compose these functions?



A Gallery of Deep Learning Models

Type of Neural Networks

- Fully connected neural networks
- Convolutional neural networks (CNNs)
- Recurrent neural networks (RNNs)
- Transformer networks
- Spiking neural networks
- …

A Gallery of Deep Learning Models

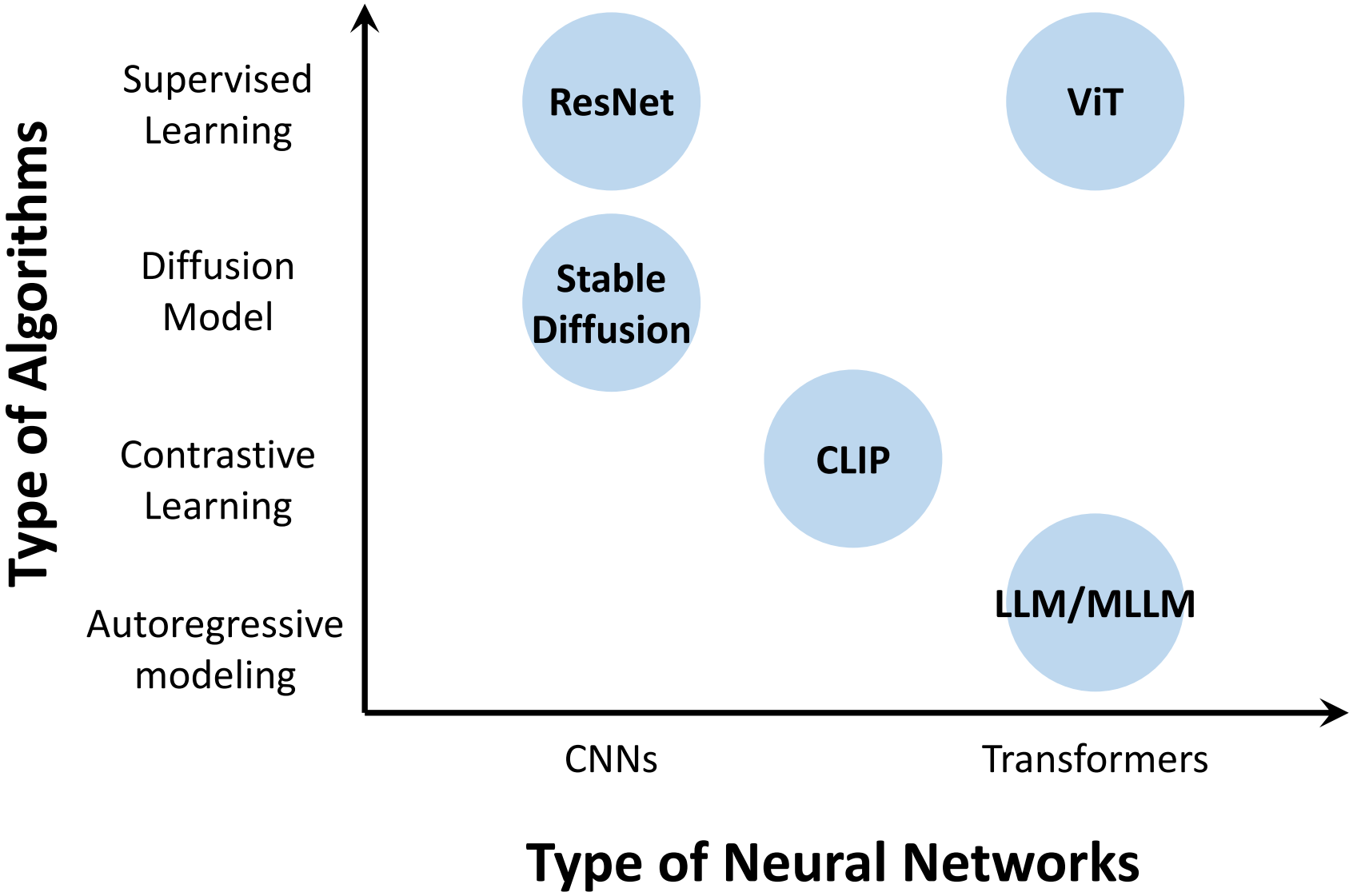
Type of Neural Networks

- Fully connected neural networks
- Convolutional neural networks (CNNs)
- Recurrent neural networks (RNNs)
- Transformer networks
- Spiking neural networks
- …

Type of Algorithms

- Supervised learning
- Autoregressive modeling
- Generative Adversarial Networks (GAN)
- Diffusion model
- Variational Autoencoder (VAE)
- Contrastive learning
- …

A Gallery of Deep Learning Models



Neural Networks

An example and a brief tutorial

Neural Networks

Assumptions (for the next few slides):

- The input image is vectorized (disregard the spatial layout of pixels)
- The target output is a categorical label (*i.e.*, classification)

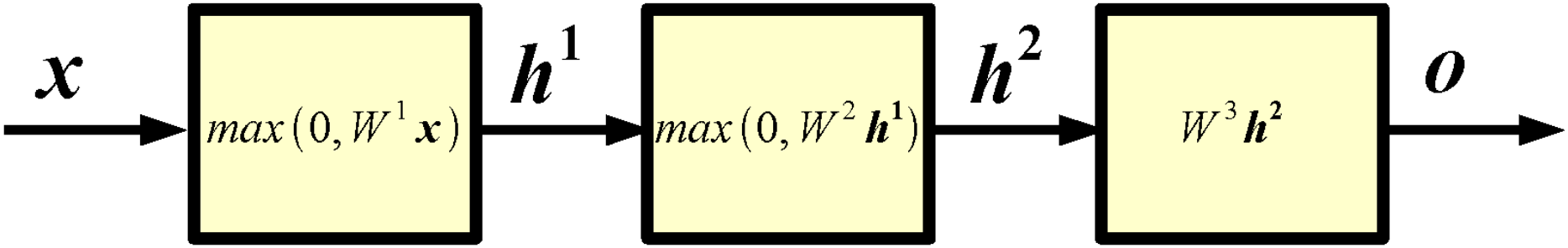
Question: what class of functions shall we consider to map the input into the output?

Answer: composition of simpler functions.

Follow-up questions: Why not a linear combination? What are the "simpler" functions? What is the interpretation?

Answer: later...

Neural Networks: Example



x input vectorized image

h^1 1-st layer hidden units (*i.e.*, feature / embedding)

h^2 2-nd layer hidden units (*i.e.*, feature / embedding)

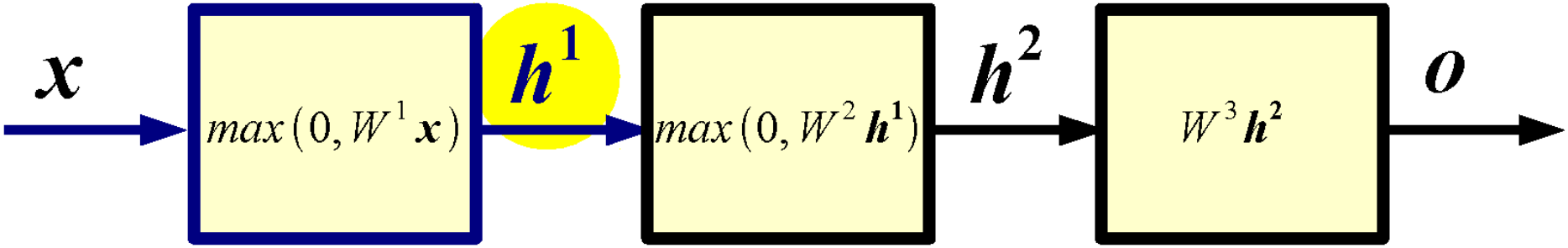
o output categorical prediction

Example of a 2 hidden layer neural network
(or a 4 layer network, counting also input and output)

Forward Propagation

Def.: Forward propagation is the process of computing the output of the network given its input.

Forward Propagation



$\boldsymbol{x} \in R^D \quad W^1 \in R^{N_1 \times D} \quad \boldsymbol{b}^1 \in R^{N_1} \quad \boldsymbol{h}^1 \in R^{N_1}$

$$\boldsymbol{h}^1 = \max(0, W^1 \boldsymbol{x} + \boldsymbol{b}^1)$$

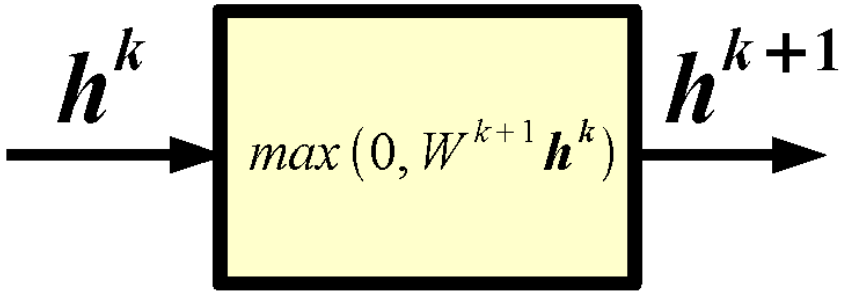
W^1 1-st layer weight matrix or weights

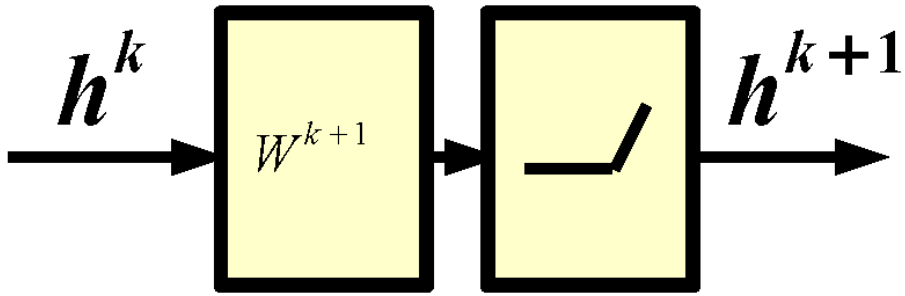
$\boldsymbol{b}^1$ 1-st layer biases

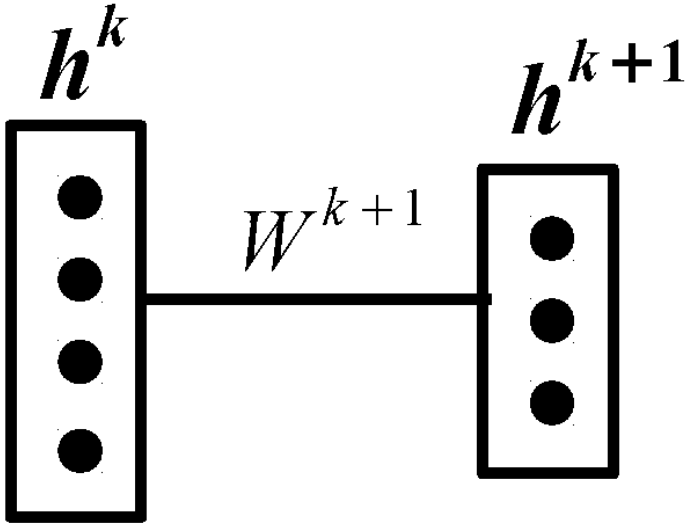
The non-linear function $u = \max(0, x)$ is call ReLU.
Each output hidden unit takes as input all the units at the previous layer; such layer is called “fully connected”

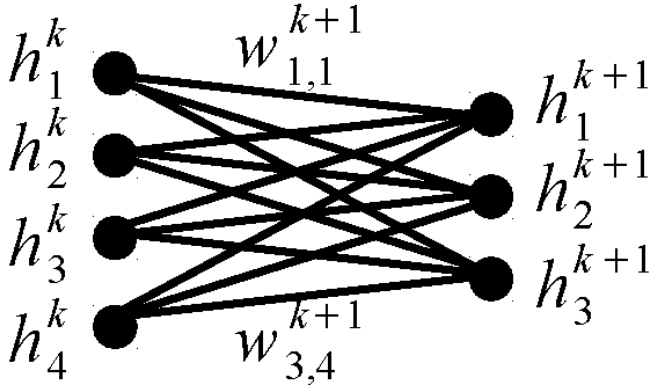
Ranzato

Alternative Graphical Representation



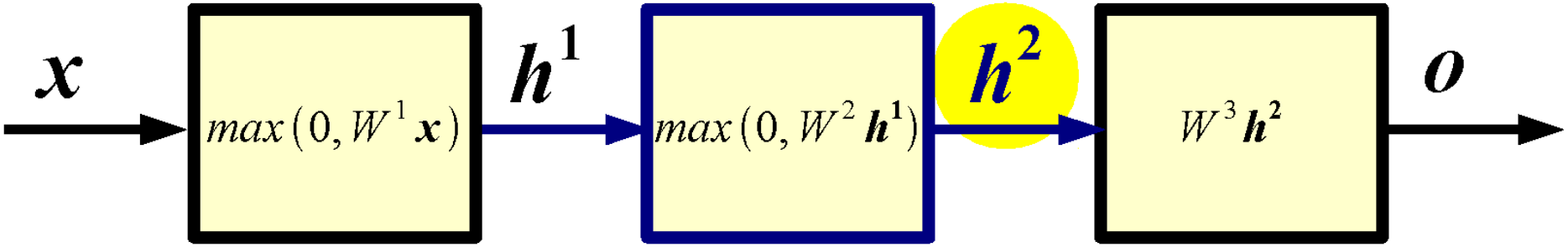






Ranzato

Forward Propagation



$\boldsymbol{h}^1 \in R^D \quad W^2 \in R^{N_2 \times N_1} \quad \boldsymbol{b}^2 \in R^{N_2} \quad \boldsymbol{h}^2 \in R^{N_2}$

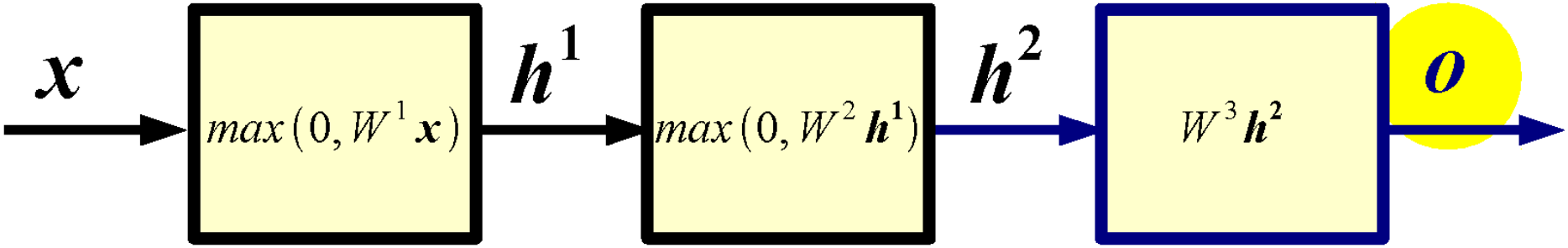
$$\boldsymbol{h}^2 = \max(0, W^1 \boldsymbol{h}^1 + \boldsymbol{b}^2)$$

W^2 2-nd layer weight matrix or weights

$\boldsymbol{b}^2$ 2-nd layer biases

Ranzato

Forward Propagation



$\boldsymbol{h}^2 \in R^{N_2}$ $W^3 \in R^{C \times N_2}$ $\boldsymbol{b}^3 \in R^C$ $\boldsymbol{o} \in R^C$

$$o = \sigma(W^3 \boldsymbol{h}^2)$$

W^3 3-rd layer weight matrix or weights

$\boldsymbol{b}^3$ 3-rd layer biases

σ an output normalization function (more later …)

Ranzato

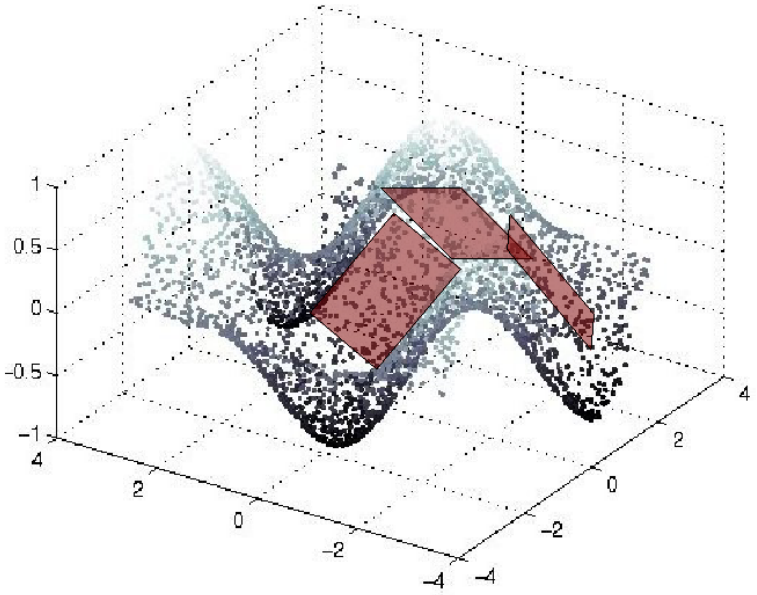
Interpretation

Question: Why can't the mapping between layers be linear?

Answer: Because composition of linear functions is a linear function. Neural network would reduce to a one layer linear model.

Question: What do ReLU layers do?

Answer: Piece-wise linear tiling: mapping is locally linear.



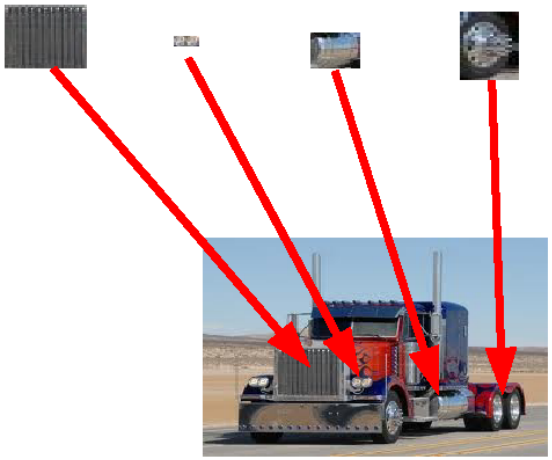
Ranzato

Interpretation

Question: Why do we need many layers?

Answer: To build a (potentially) **distributed** and **hierarchical** representations of visual concepts suitable for recognition

[0 0 **1** 0 0 0 0 **1** 0 0 **1** **1** 0 0 **1** 0 . . .] truck feature



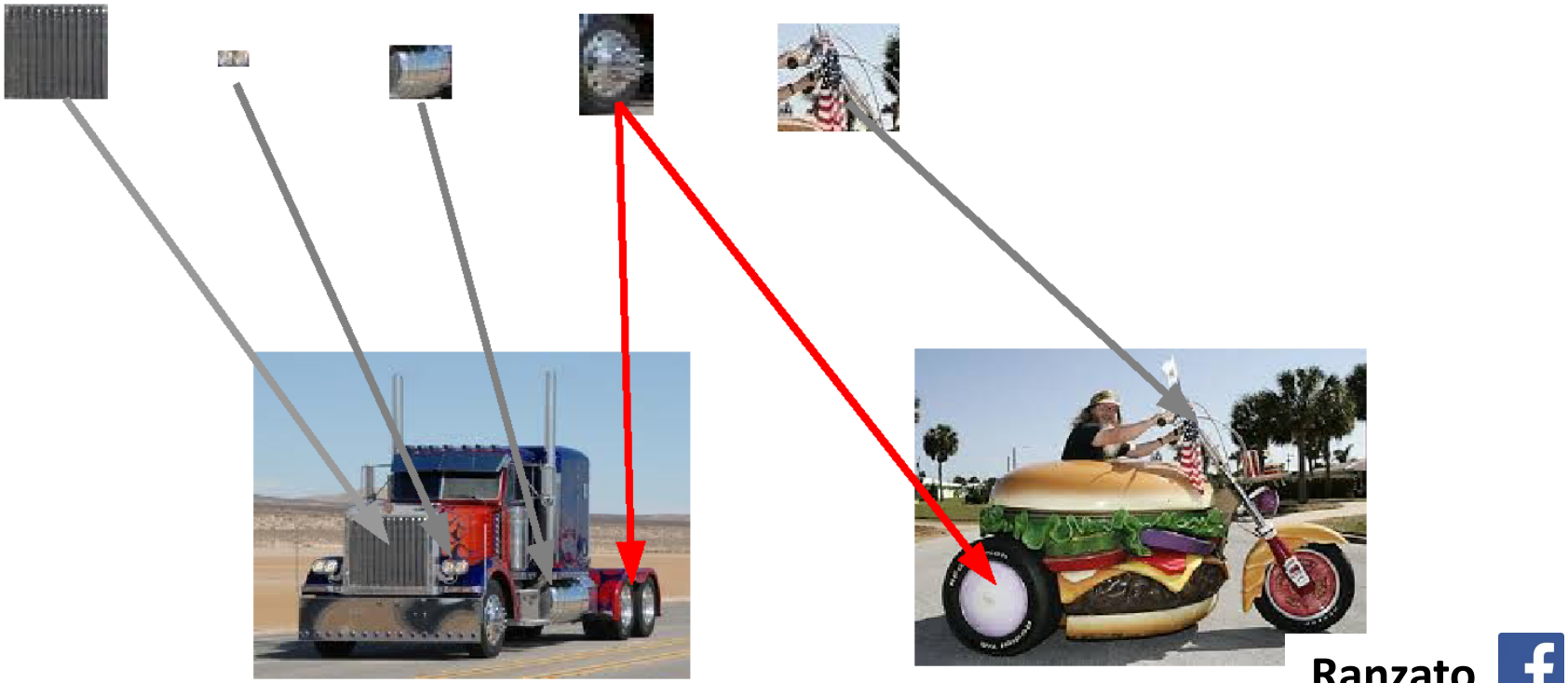
Exponentially more efficient than 1-of-N representations

Ranzato

Interpretation

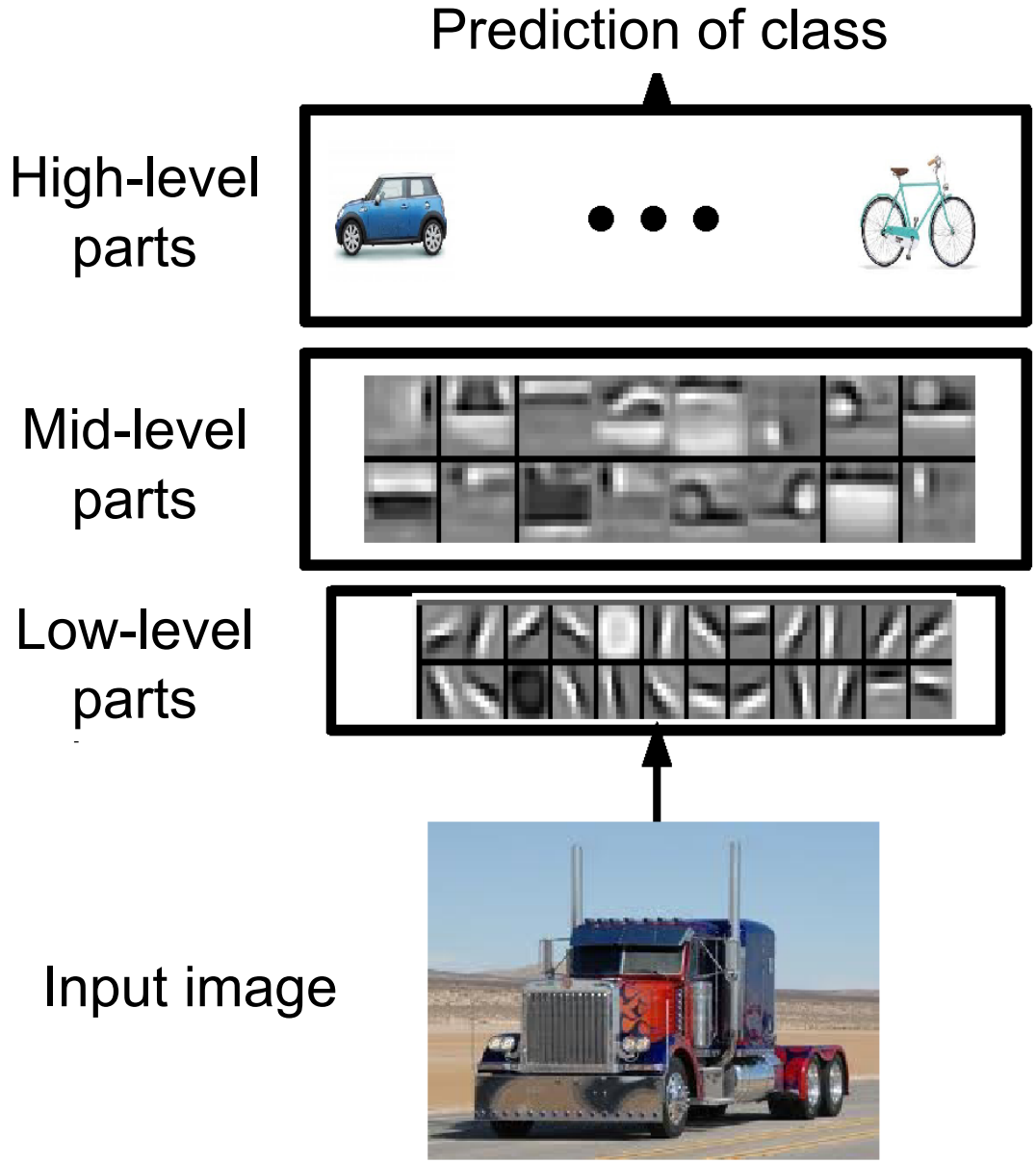
[1 1 0 0 0 1 0 **1** 0 0 0 0 1 1 0 1…] motorbike

[0 0 1 0 0 0 0 **1** 0 0 1 1 0 0 1 0 …] truck



Ranzato

Interpretation



Lee et al. “Convolutional DBN's ...” ICML 2009

Ranzato

Interpretation

Question: What does a hidden unit do?

Answer: Think about a feature detector (learned from data).

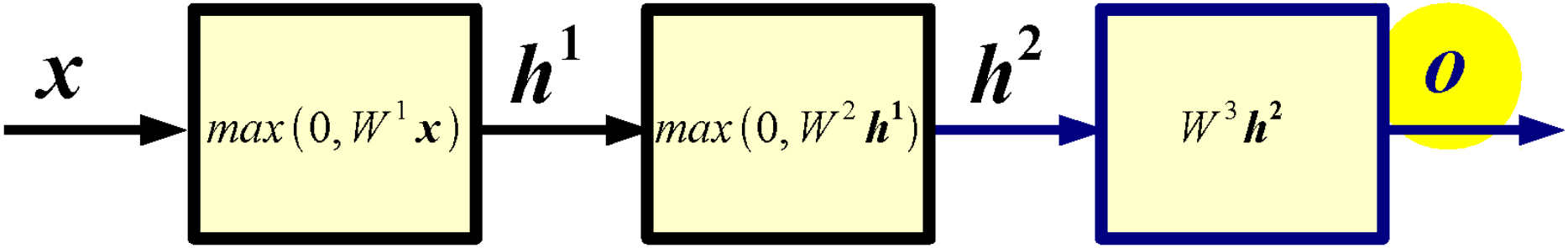
Question: How many layers? How many hidden units?

Answer: Empirically determined via cross-validation or hyper-parameter search. A good design must balance the model capacity (depth and width) and computational efficiency.

Question: How do I set the weight matrices?

Answer: **W**eight matrices and biases are learned from data.
First, we need to define a measure of quality of the current mapping.
Then, we need to define a procedure to adjust the parameters.

Forward Propagation



$\boldsymbol{h}^2 \in R^{N_2}$ $W^3 \in R^{C \times N_2}$ $\boldsymbol{b}^3 \in R^C$ $\boldsymbol{o} \in R^C$

$$o = \sigma(W^3 \boldsymbol{h}^2)$$

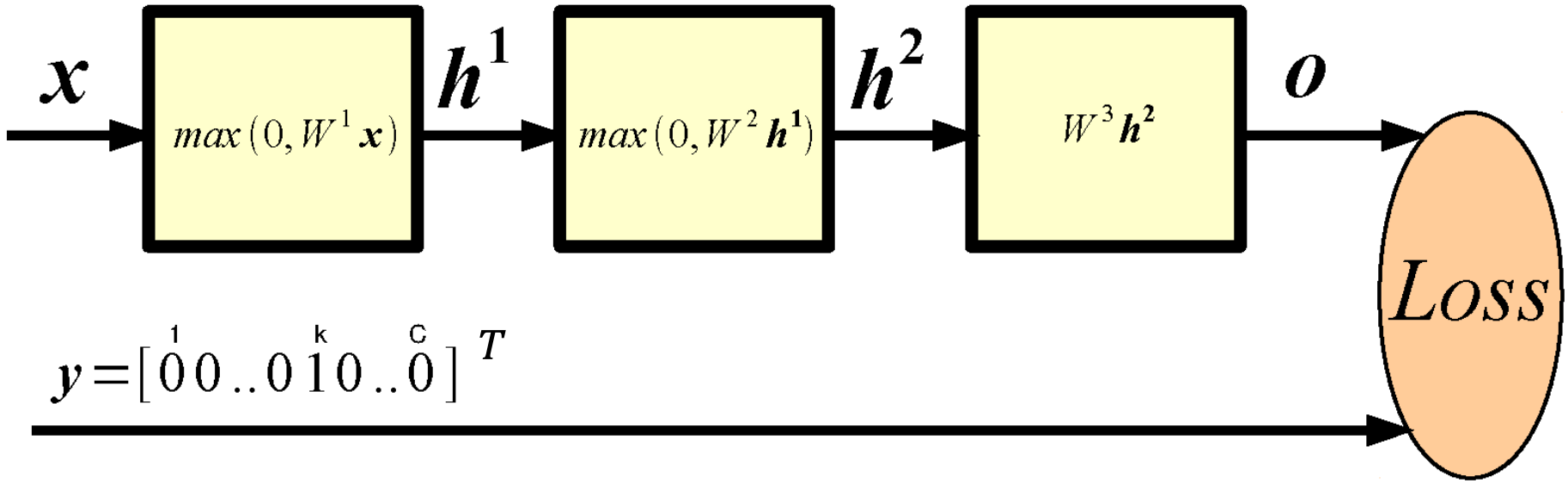
W^3 3-rd layer weight matrix or weights

$\boldsymbol{b}^3$ 3-rd layer biases

σ an output normalization function (softmax)

Ranzato

How Good is a Network?



Probability of class K given input via Softmax normalization

$$p(c_k = 1|x) = \frac{\exp(o_k)}{\sum_{j=1}^{C} \exp(o_j)}$$

Loss: per-sample negative log-likelihood (good for classification of small number of classes)

$$L(\boldsymbol{x}, y; \theta) = -\sum_{j=1}^{C} y_j \log p(c_j|x)$$

Ranzato

Training

Learning consists of minimizing the loss (plus some regularization term) w.r.t. model parameters over the whole training set

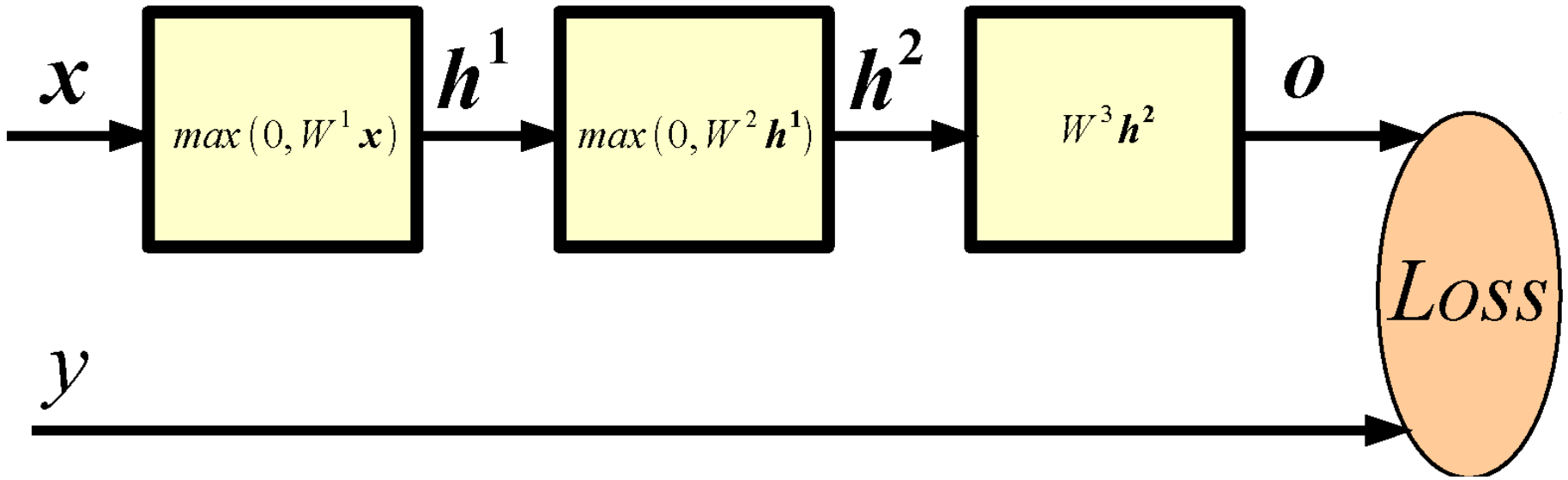
$$\theta^* = argmin_\theta \sum_{n=1}^{N} L(x_n, y_n;\ \theta)$$

Question: How to minimize the loss (a complicated function of the model parameters)?

Answer: Chain rule, i.e., **Backpropagation!** That is the procedure to compute the gradients of the loss w.r.t. model parameters in a multi-layer neural network

Rumelhart et al. "Learning internal representations by back-propagating.." Nature 1986

Key Idea: Wiggle To Decrease Loss



Let us say we want to decrease the loss by adjusting $W^1_{i,j}$

We could consider a very small perturbation ϵ=1e-6 and compute

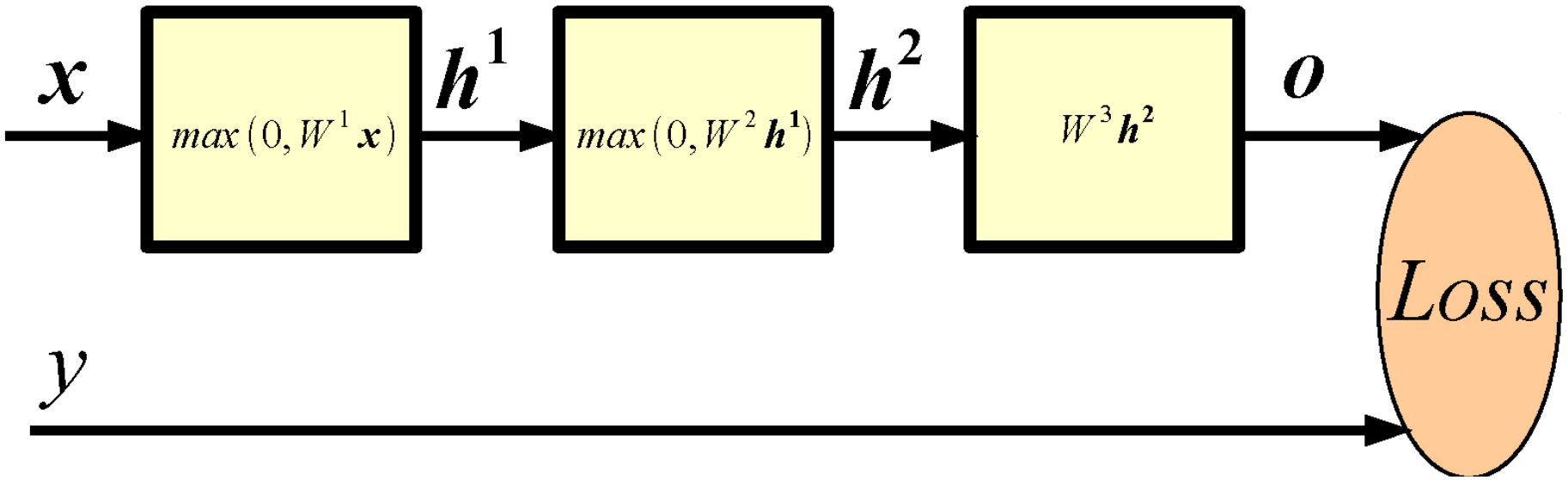
$$L(\boldsymbol{x}, y; \boldsymbol{\theta})$$

$$L(\boldsymbol{x}, y; \boldsymbol{\theta} \setminus W^1_{i,j}, W^1_{i,j} + \epsilon)$$

Then, update:

$$W^1_{i,j} \leftarrow W^1_{i,j} + \epsilon \, sgn\left(L(\boldsymbol{x}, y; \boldsymbol{\theta}) - L(\boldsymbol{x}, y; \boldsymbol{\theta} \setminus W^1_{i,j}, W^1_{i,j} + \epsilon)\right)$$

Key Idea: Wiggle To Decrease Loss

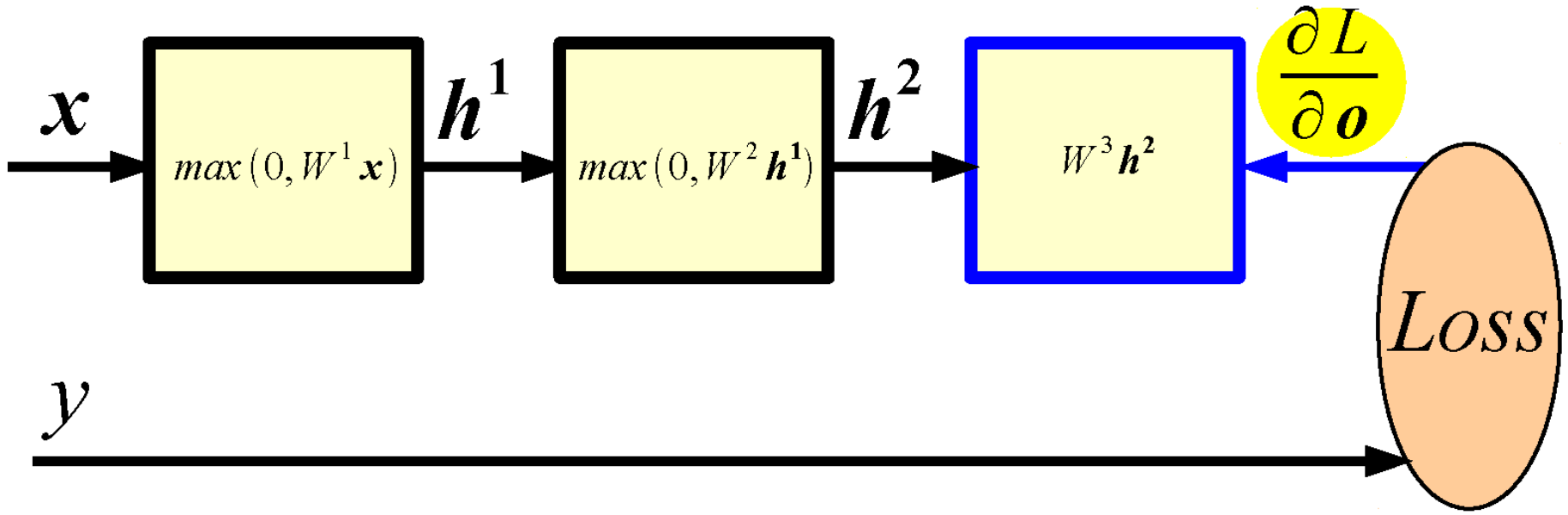


Let us say we want to decrease the loss by adjusting $W^1_{i,j}$

We could consider a very small perturbation ϵ=1e-6 and compute

$$W^1 \leftarrow W^1 - \epsilon \frac{\partial L}{\partial W^1}$$

Backward Propagation



Ranzato

Derivative w.r.t. Input of Softmax

$$p(c_k = 1|\boldsymbol{x}) = \frac{\exp(o_k)}{\sum_{j=1}^{C} \exp(o_j)}$$

$$L(\boldsymbol{x}, y; \theta) = -\sum_{j=1}^{C} y_j \log p(c_j|\boldsymbol{x}) \qquad T$$

By substituting the first formulat in the second, and taking the derivative w.r.t. $\boldsymbol{o}$, we get

$$\frac{\partial L}{\partial \boldsymbol{o}} = (p(c|\boldsymbol{x}) - y)^T \in R^{1\times c}$$

Derivative of $f: R^n \rightarrow R^m$:
Jacobian matrix of size $m \times n$

Can you prove this?

Ranzato

Let $p_j = p(c_j|\boldsymbol{x}) = \dfrac{\exp(o_j)}{\sum_{k=1}^{C}\exp(o_k)}$

$$L = -\sum_{j=1}^{C} y_j \log(p_j)$$

Sum to 1 Constant for all y_j

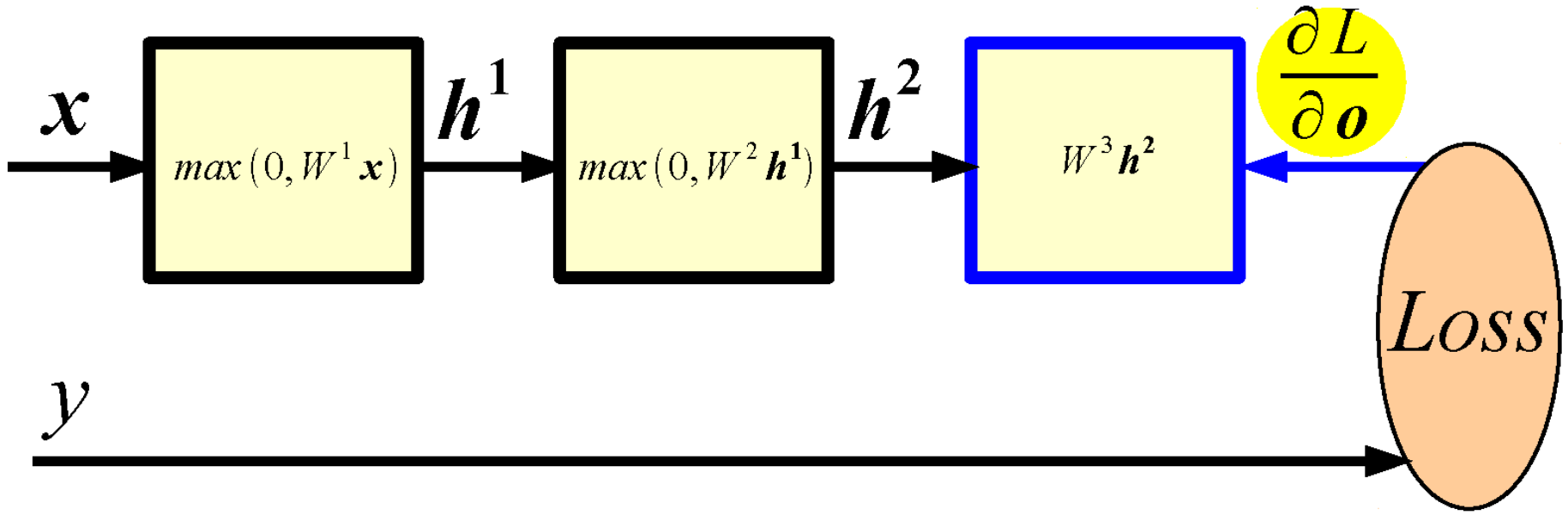
$$= -\sum_{j=1}^{C} y_j \log(\exp(o_j)) + \sum_{j=1}^{C} y_j \log\left(\sum_{k=1}^{C}\exp(o_k)\right)$$

$$= -\sum_{j=1}^{C} y_j o_j + \log\left(\sum_{k=1}^{C}\exp(o_k)\right)$$

$$\frac{\partial L}{\partial o_j} = -y_j + \frac{\exp(o_j)}{\sum_{k=1}^{C}\exp(o_k)} = p_j - y_j$$

$$\Rightarrow \frac{\partial L}{\partial \boldsymbol{o}} = (p(c|\boldsymbol{x}) - y)^T \in R^{1\times c}$$

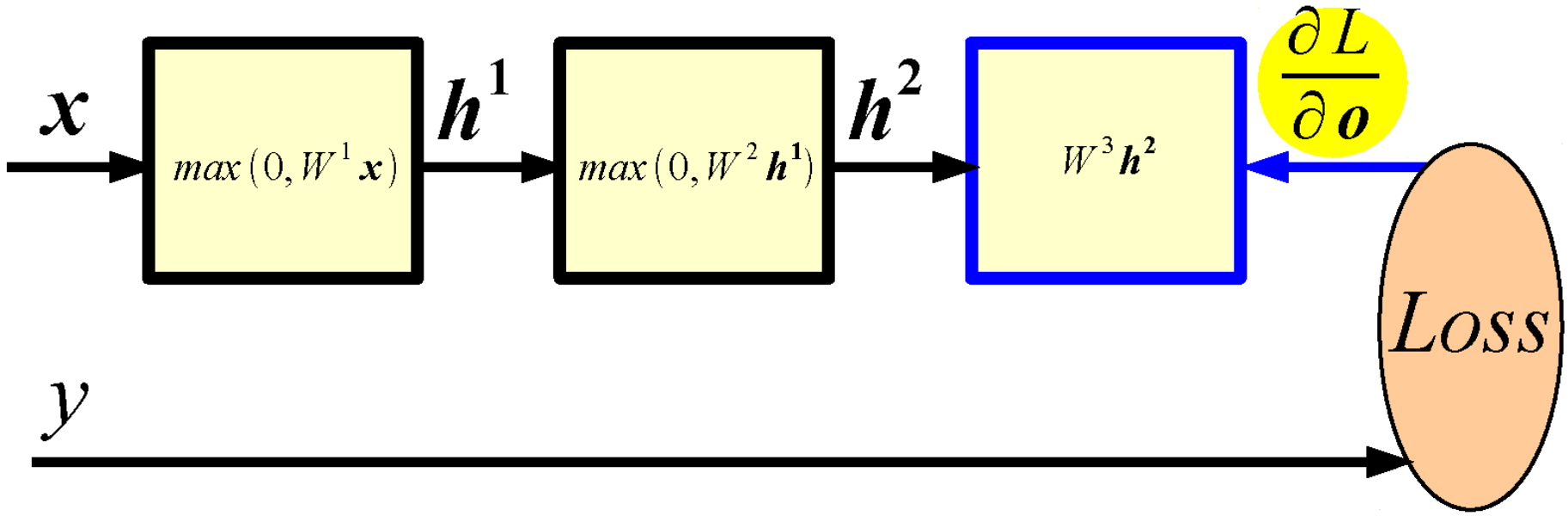
Backward Propagation



Given $\partial L / \partial \boldsymbol{o}$ and assuming we can easily compute the Jacobian of each module, we have:

$$\frac{\partial L}{\partial W^3} = \frac{\partial L}{\partial \boldsymbol{o}} \frac{\partial \boldsymbol{o}}{\partial W^3} \qquad \frac{\partial L}{\partial \boldsymbol{h}^2} = \frac{\partial L}{\partial \boldsymbol{o}} \frac{\partial \boldsymbol{o}}{\partial \boldsymbol{h}^2}$$

Backward Propagation



Given $\partial L / \partial \boldsymbol{o}$ and assuming we can easily compute the Jacobian of each module, we have:

$$\frac{\partial L}{\partial \boldsymbol{h}^2} = \frac{\partial L}{\partial \boldsymbol{o}} \frac{\partial \boldsymbol{o}}{\partial \boldsymbol{h}^2}$$

$$\frac{\partial L}{\partial \boldsymbol{h}^2} = (p(c|\boldsymbol{x}) - y)^T W^3$$

Ranzato

Consider $\boldsymbol{o} = W\boldsymbol{h}$ with $\boldsymbol{o} \in R^n, \boldsymbol{h} \in R^m, and\ W \in R^{n \times m}$

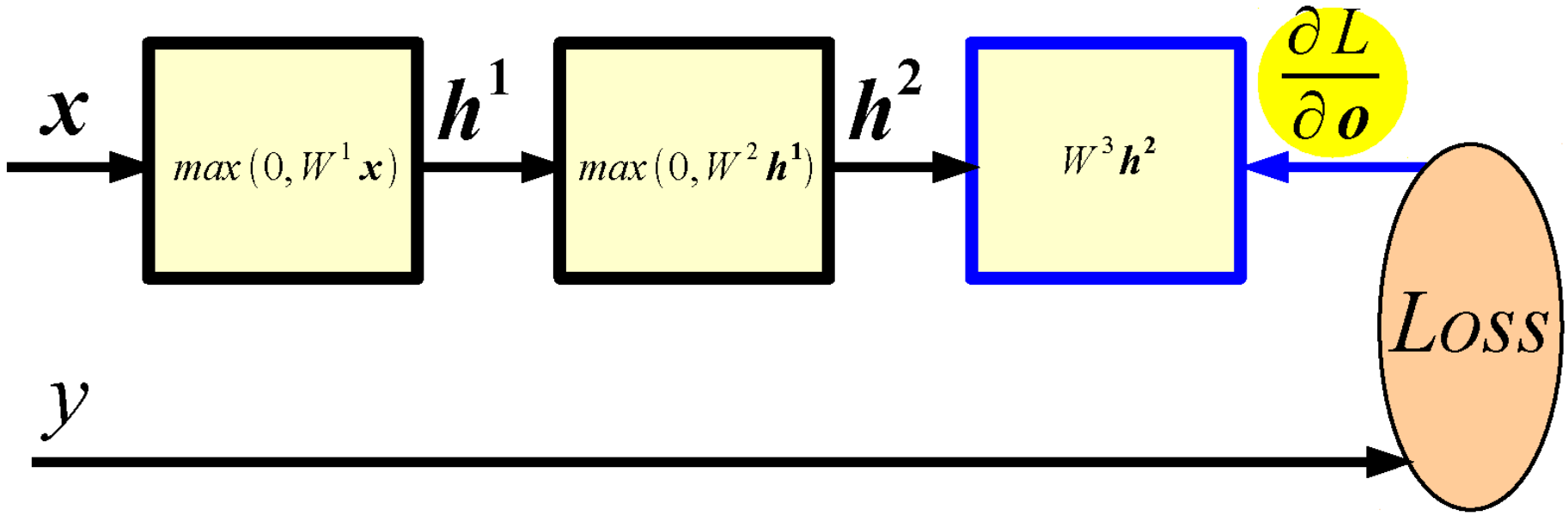
Denote $W = \begin{bmatrix} \boldsymbol{w}_1^T \\ \boldsymbol{w}_2^T \\ \vdots \\ \boldsymbol{w}_m^T \end{bmatrix}$ with $\boldsymbol{w}_i^T$ as the *i*-th row

$$\boldsymbol{o} = \begin{bmatrix} o_1 \\ o_2 \\ \vdots \\ o_n \end{bmatrix} = \begin{bmatrix} \boldsymbol{w}_1^T\boldsymbol{h} \\ \boldsymbol{w}_2^T\boldsymbol{h} \\ \vdots \\ \boldsymbol{w}_m^T\boldsymbol{h} \end{bmatrix} \Rightarrow \quad \frac{\partial o_i}{\partial h_j} = w_{ij}$$

Element at row *i* and column *j*

$$\Rightarrow \quad \frac{\partial o_i}{\partial \boldsymbol{h}} = \boldsymbol{w}_i^T \quad \Rightarrow \quad \frac{\partial \boldsymbol{o}}{\partial \boldsymbol{h}} = W$$

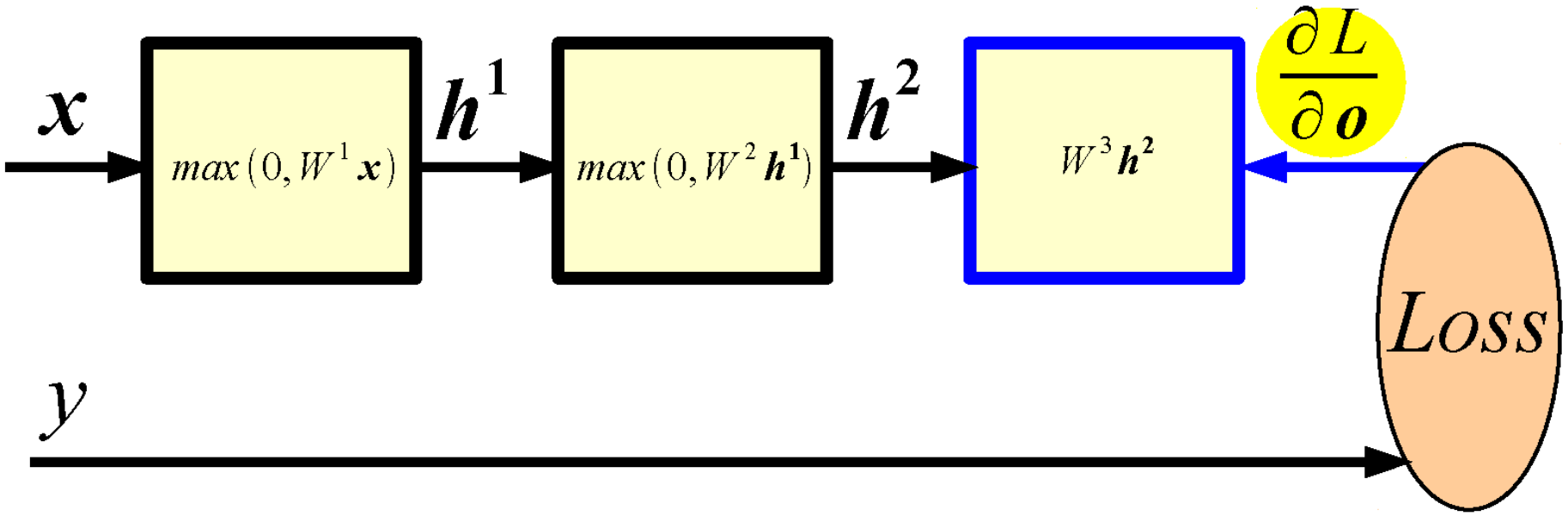
Backward Propagation



Given $\partial L / \partial \boldsymbol{o}$ and assuming we can easily compute the Jacobian of each module, we have:

$$\frac{\partial L}{\partial W^3} = \frac{\partial L}{\partial \boldsymbol{o}} \frac{\partial \boldsymbol{o}}{\partial W^3}$$

Backward Propagation



Given $\partial L / \partial \boldsymbol{o}$ and assuming we can easily compute the Jacobian of each module, we have:

$$\frac{\partial L}{\partial W^3} = \frac{\partial L}{\partial \boldsymbol{o}} \frac{\partial \boldsymbol{o}}{\partial W^3}$$

$$\frac{\partial L}{\partial W^3} = (p(c|\boldsymbol{x}) - y){h^2}^T$$

Ranzato

Consider $\boldsymbol{o} = W\boldsymbol{h}$ with $\boldsymbol{o} \in R^n, \boldsymbol{h} \in R^m, and\ W \in R^{n \times m}$

Denote $W = \begin{bmatrix} \boldsymbol{w}_1^T \\ \boldsymbol{w}_2^T \\ \vdots \\ \boldsymbol{w}_n^T \end{bmatrix}$ with $\boldsymbol{w}_i^T$ as the *i*-th row

Let $\mathbf{0}$ be an all-zero column vector in R^m

$$\boldsymbol{o} = \begin{bmatrix} o_1 \\ o_2 \\ \vdots \\ o_n \end{bmatrix} = \begin{bmatrix} \boldsymbol{w}_1^T \boldsymbol{h} \\ \boldsymbol{w}_2^T \boldsymbol{h} \\ \vdots \\ \boldsymbol{w}_n^T \boldsymbol{h} \end{bmatrix}$$

Consider $\boldsymbol{o} = W\boldsymbol{h}$ with $\boldsymbol{o} \in R^n, \boldsymbol{h} \in R^m, and\ W \in R^{n \times m}$

Denote $W = \begin{bmatrix} \boldsymbol{w}_1^T \\ \boldsymbol{w}_2^T \\ \vdots \\ \boldsymbol{w}_n^T \end{bmatrix}$ with $\boldsymbol{w}_i^T$ as the *i*-th row

Let $\mathbf{0}$ be an all-zero column vector in R^m

$$\begin{bmatrix} o_1 \\ o_2 \\ \vdots \\ o_n \end{bmatrix} = \begin{bmatrix} \boldsymbol{w}_1^T\boldsymbol{h} \\ \boldsymbol{w}_2^T\boldsymbol{h} \\ \vdots \\ \boldsymbol{w}_n^T\boldsymbol{h} \end{bmatrix} = \begin{bmatrix} \boldsymbol{h}^T & \mathbf{0}^T & \dots & \mathbf{0}^T \\ \mathbf{0}^T & \boldsymbol{h}^T & & \\ \dots & \dots & \dots & \dots \\ \mathbf{0}^T & \mathbf{0}^T & \mathbf{0}^T & \boldsymbol{h}^T \end{bmatrix} \begin{bmatrix} \boldsymbol{w}_1 \\ \boldsymbol{w}_2 \\ \dots \\ \boldsymbol{w}_n \end{bmatrix} \triangleq H\ vec(W)$$

n rows,
n*m columns (n blocks)

vectorized version of *W*
with n*m elements

Consider $\boldsymbol{o} = W\boldsymbol{h}$ with $\boldsymbol{o} \in R^n, \boldsymbol{h} \in R^m, and\ W \in R^{n \times m}$

Denote $W = \begin{bmatrix} \boldsymbol{w}_1^T \\ \boldsymbol{w}_2^T \\ \vdots \\ \boldsymbol{w}_n^T \end{bmatrix}$ with $\boldsymbol{w}_i^T$ as the *i*-th row

Let $\mathbf{0}$ be an all-zero column vector in R^m

$$\begin{bmatrix} o_1 \\ o_2 \\ \vdots \\ o_n \end{bmatrix} = \begin{bmatrix} \boldsymbol{w}_1^T\boldsymbol{h} \\ \boldsymbol{w}_2^T\boldsymbol{h} \\ \vdots \\ \boldsymbol{w}_n^T\boldsymbol{h} \end{bmatrix} = \begin{bmatrix} \boldsymbol{h}^T & \mathbf{0}^T & \dots & \mathbf{0}^T \\ \mathbf{0}^T & \boldsymbol{h}^T & & \\ \dots & \dots & \dots & \dots \\ \mathbf{0}^T & \mathbf{0}^T & \mathbf{0}^T & \boldsymbol{h}^T \end{bmatrix} \begin{bmatrix} \boldsymbol{w}_1 \\ \boldsymbol{w}_2 \\ \dots \\ \boldsymbol{w}_n \end{bmatrix} \triangleq H\ vec(W)$$

This (direct linear transformation) holds, as each element

$$o_i = \boldsymbol{w}_i^T\boldsymbol{h} = \boldsymbol{h}^T\boldsymbol{w}_i$$

Consider $\boldsymbol{o} = W\boldsymbol{h}$ with $\boldsymbol{o} \in R^n, \boldsymbol{h} \in R^m, and\ W \in R^{n \times m}$

Let $\mathbf{0}$ be an all-zero column vector in R^m

n rows,
n*m columns (n blocks)

vectorized version of W
with n*m elements

$$\boldsymbol{o} = \begin{bmatrix} o_1 \\ o_2 \\ \vdots \\ o_n \end{bmatrix} = \begin{bmatrix} \boldsymbol{h}^T & \mathbf{0}^T & \dots & \mathbf{0}^T \\ \mathbf{0}^T & \boldsymbol{h}^T & & \\ \dots & \dots & \dots & \dots \\ \mathbf{0}^T & \mathbf{0}^T & \mathbf{0}^T & \boldsymbol{h}^T \end{bmatrix} \begin{bmatrix} \boldsymbol{w}_1 \\ \boldsymbol{w}_2 \\ \dots \\ \boldsymbol{w}_n \end{bmatrix} \triangleq H\ vec(W)$$

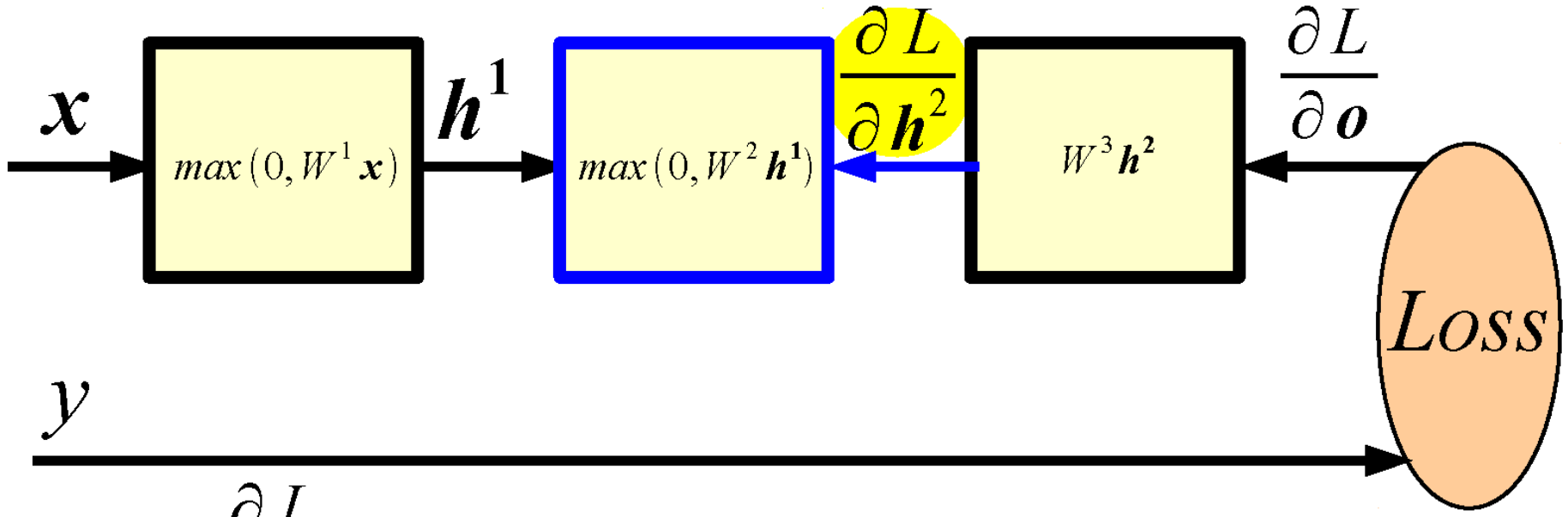
Kronecker Product

Recall that $\dfrac{\partial A\boldsymbol{x}}{\partial \boldsymbol{x}} = \mathrm{A} \quad \Rightarrow \dfrac{\partial \boldsymbol{o}}{\partial vec(W)} = H = I_n \otimes h^T$

Finally, we verify $(p(c|\boldsymbol{x}) - \boldsymbol{y})^T H = vec((p(c|\boldsymbol{x}) - \boldsymbol{y})\, \boldsymbol{h}^T)$

(Hint: simply check every element in the result vector)

Backward Propagation

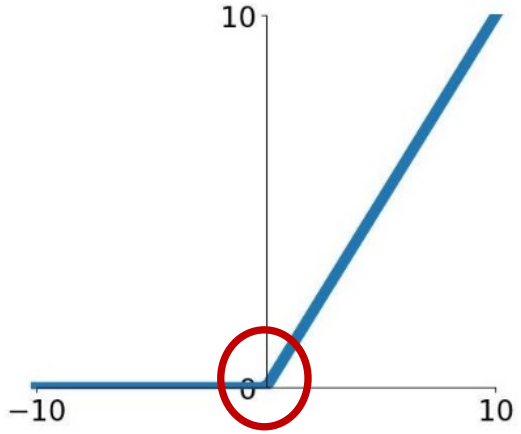


Given $\frac{\partial L}{\partial \boldsymbol{h}^2}$ we can compute now:

$$\frac{\partial L}{\partial W^2} = \frac{\partial L}{\partial \boldsymbol{h}^2}\frac{\partial \boldsymbol{h}^2}{\partial W^2} \qquad \frac{\partial L}{\partial \boldsymbol{h}^1} = \frac{\partial L}{\partial \boldsymbol{h}^2}\frac{\partial \boldsymbol{h}^2}{\partial \boldsymbol{h}^1}$$

Ranzato

Consider $\boldsymbol{h} = \max(0, \boldsymbol{x})$ with $\boldsymbol{h} \in R^m, and\ \boldsymbol{x} \in R^m$

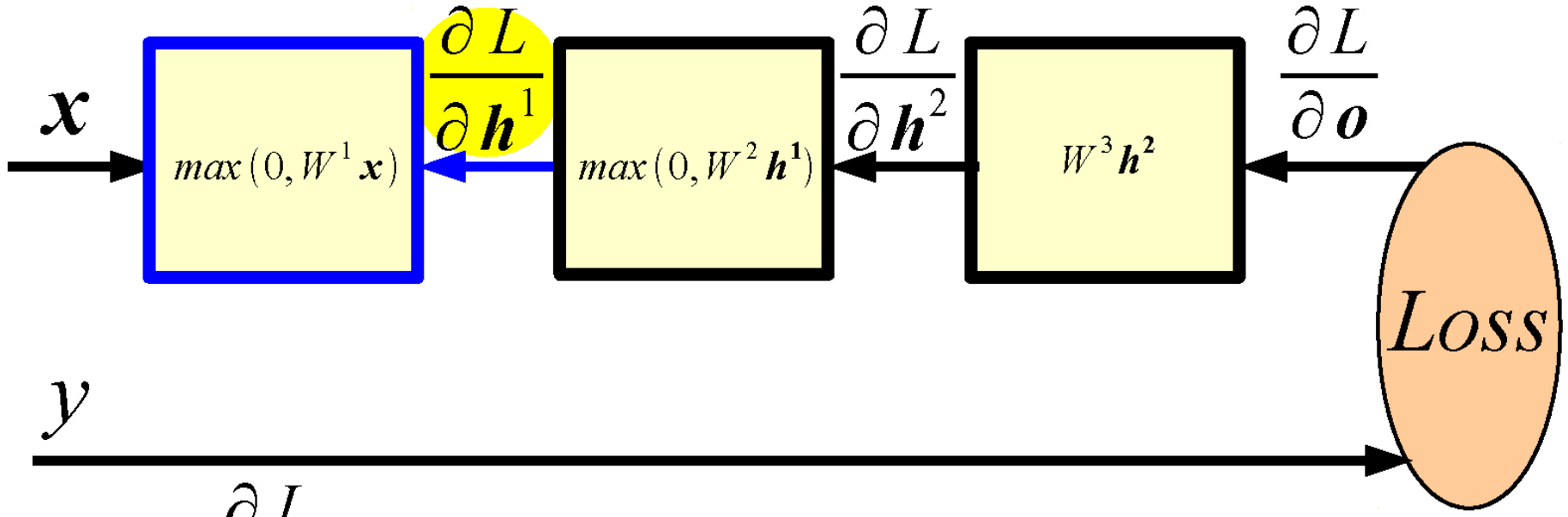


$$\frac{\partial \boldsymbol{h}}{\partial \boldsymbol{x}} = \begin{cases} 1 & \boldsymbol{x} > 0 \\ 0 & \boldsymbol{x} < 0 \\ ? & \boldsymbol{x} = 0 \end{cases}$$

Use sub-gradients by patching up the derivatives

$$\frac{\partial \boldsymbol{h}}{\partial \boldsymbol{x}} = \begin{cases} 1 & x \geq 0 \\ 0 & x < 0 \end{cases}$$

Backward Propagation



Given $\frac{\partial L}{\partial \boldsymbol{h}^1}$ we can compute now:

$$\frac{\partial L}{\partial W^1} = \frac{\partial L}{\partial \boldsymbol{h}^1} \frac{\partial \boldsymbol{h}^1}{\partial W^1}$$

Ranzato

Training as Optimization

Stochastic Gradient Descent (on mini-batches):

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \frac{\partial L}{\partial \boldsymbol{\theta}}, \eta \in (0, 1)$$ Learning Rate

Note: there are many other variants...

Training as Optimization

Repeat:

- **Randomly sample** a batch of data
- **Forward** prop the batch through the network, and compute loss
- **Backprop** to calculate the gradients
- **Update** the parameters using the gradients

Till convergence?

Training as Optimization

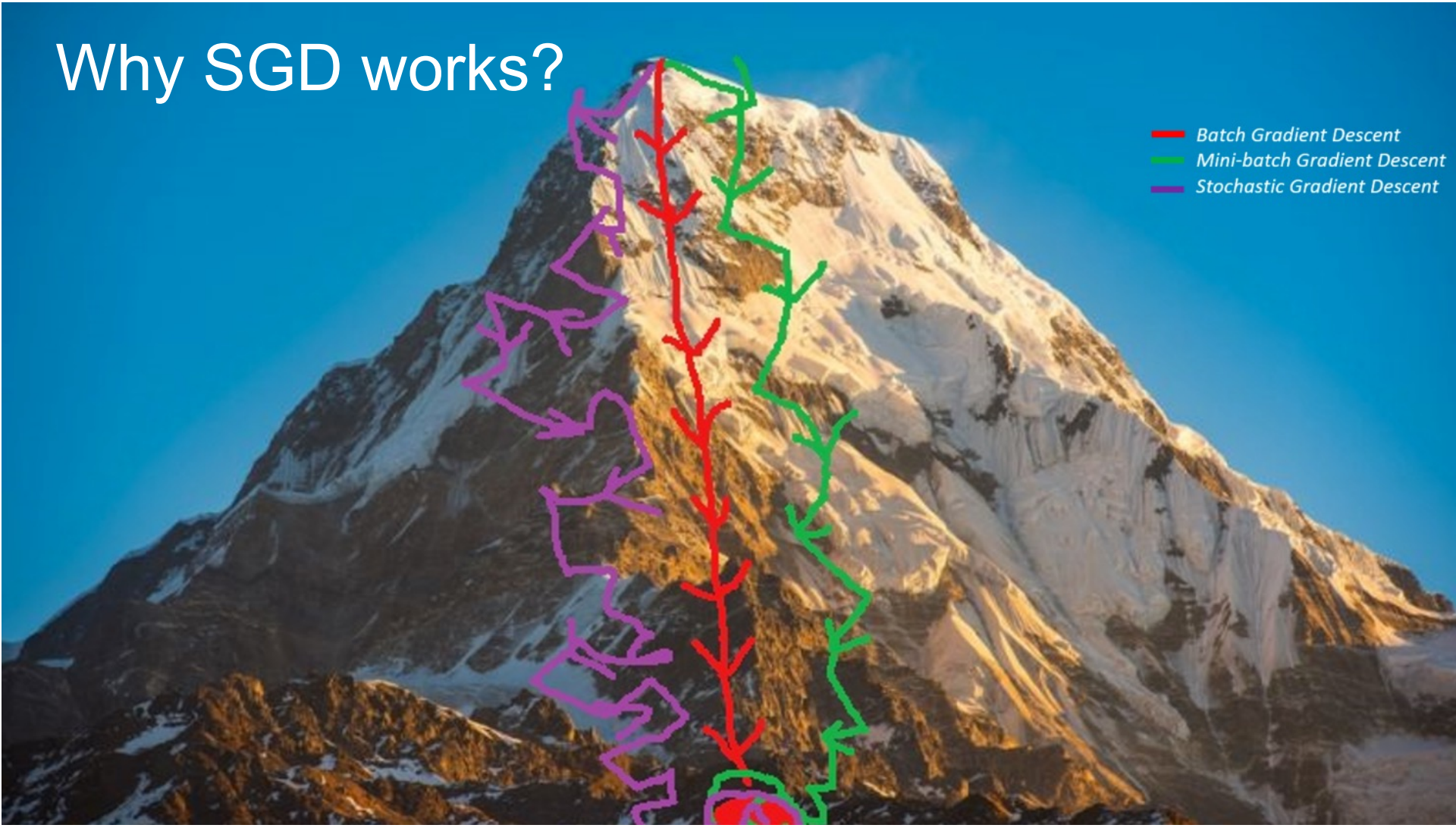


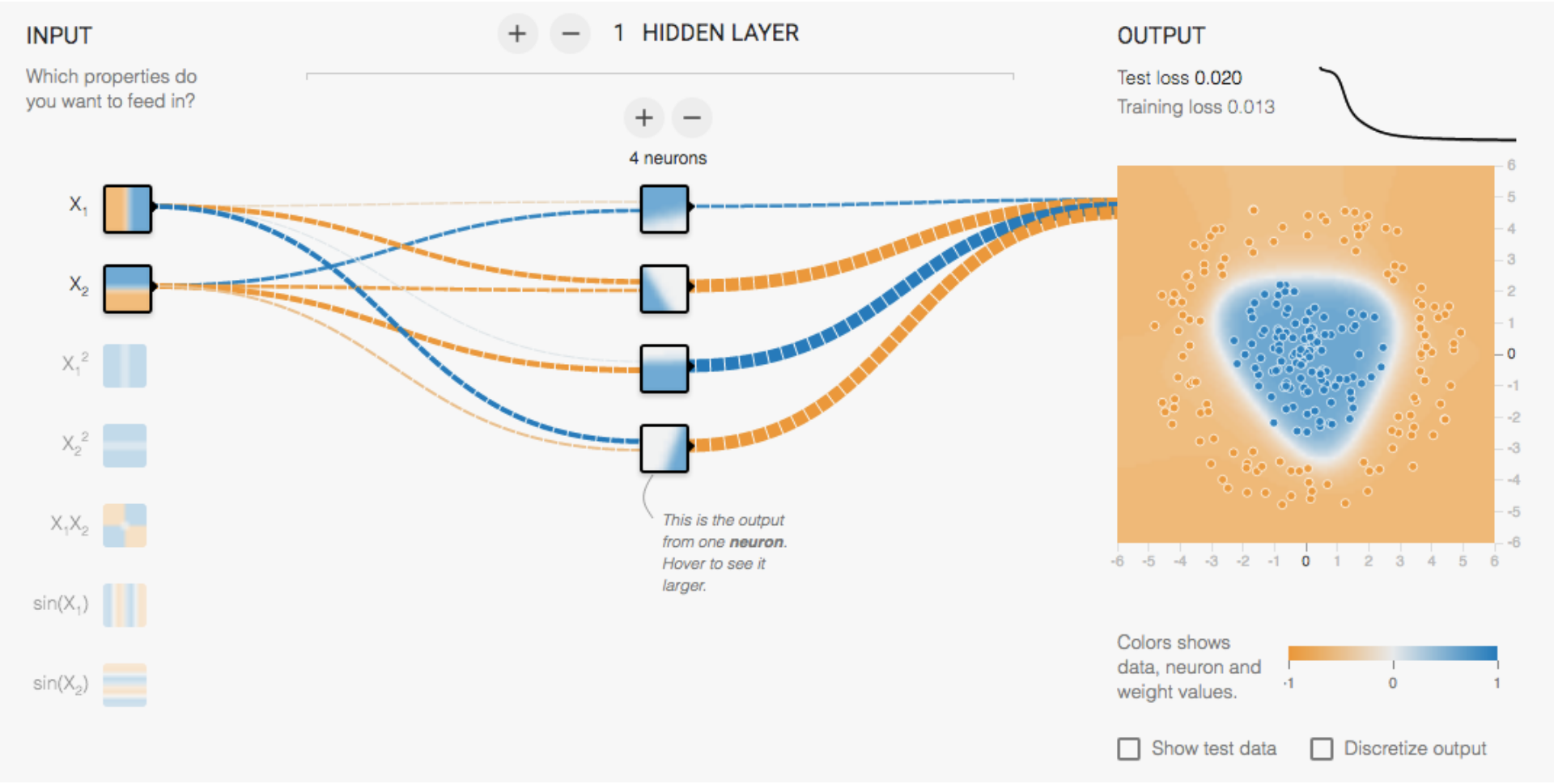
Image Source:
https://towardsdatascience.com/gradient-descent-algorithm-and-its-variants-10f652806a3

Training as Optimization

Why SGD?

- Can easily fit into memory
- Implicit regularization of learning
- Works very well in practice
  (if you know how to select the hyper-parameters)

Multi-layer Neural Network: Demo



http://playground.tensorflow.org/

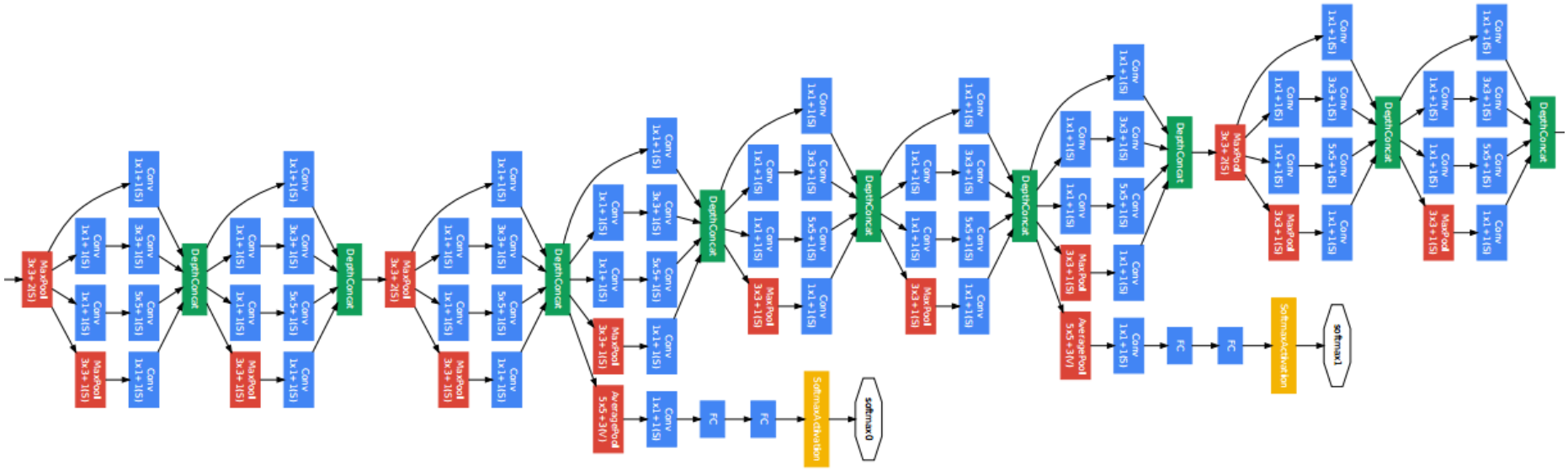
Recap: Deep Learning

- (Deep) Neural Network:
  Composing a set of (nonlinear) functions *g*

$$f(\boldsymbol{x};\boldsymbol{\theta}) = g_1(\dots g_{n-1}(g_n(\boldsymbol{x};\boldsymbol{\theta_n}),\boldsymbol{\theta_{n-1}})\dots,\boldsymbol{\theta_1})$$

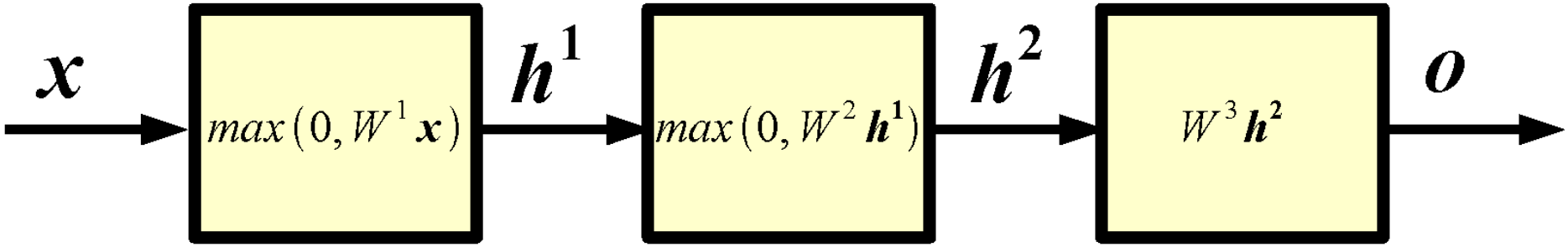
- Forward propagation: compute the output given the input and the parameters
- Backward propagation: compute the gradients of the parameters by comparing the output to a target label using a loss function
- Training: Finding parameters that minimize the average loss on the training set

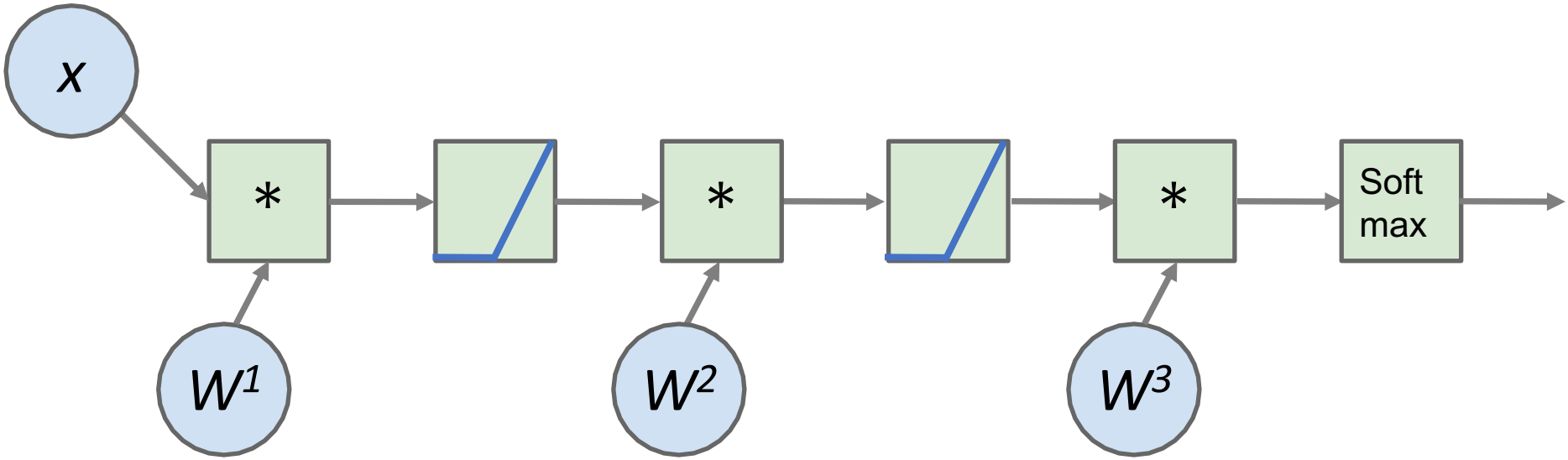
Backprop for a modern network?



Szegedy, C., Liu, W., Jia, Y., Sermanet, P., Reed, S., Anguelov, D., ... & Rabinovich, A. (2015). Going deeper with convolutions. In *CVPR* (pp. 1-9).

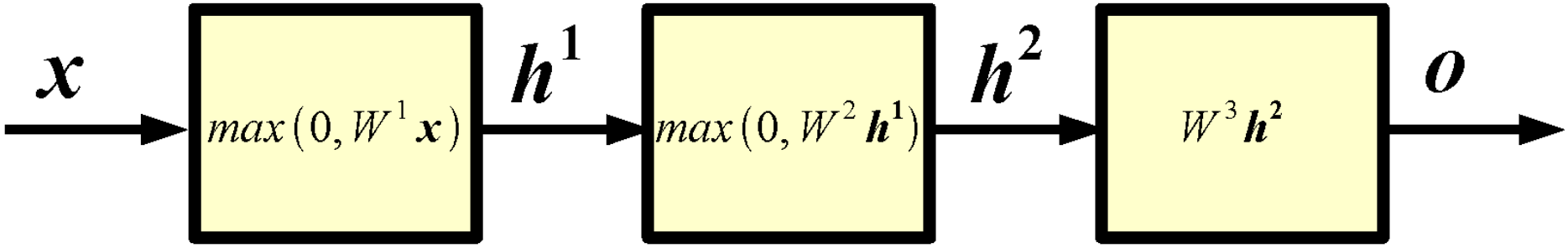
Backprop on Computational Graph

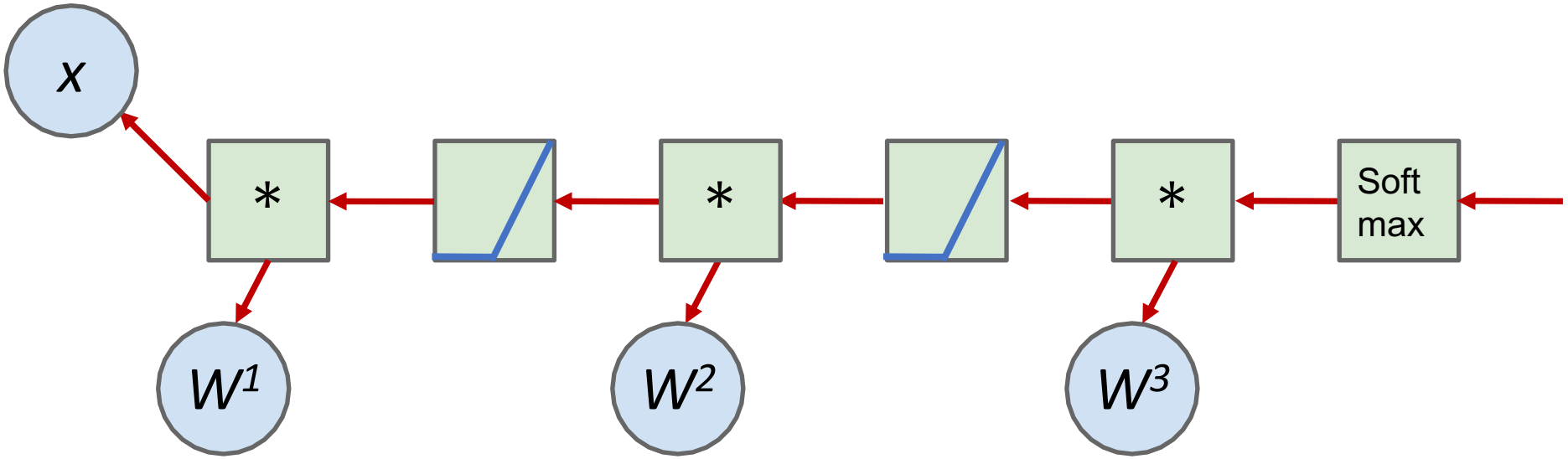




- Leaf nodes: variables (inputs or parameters)
- Other nodes: operators

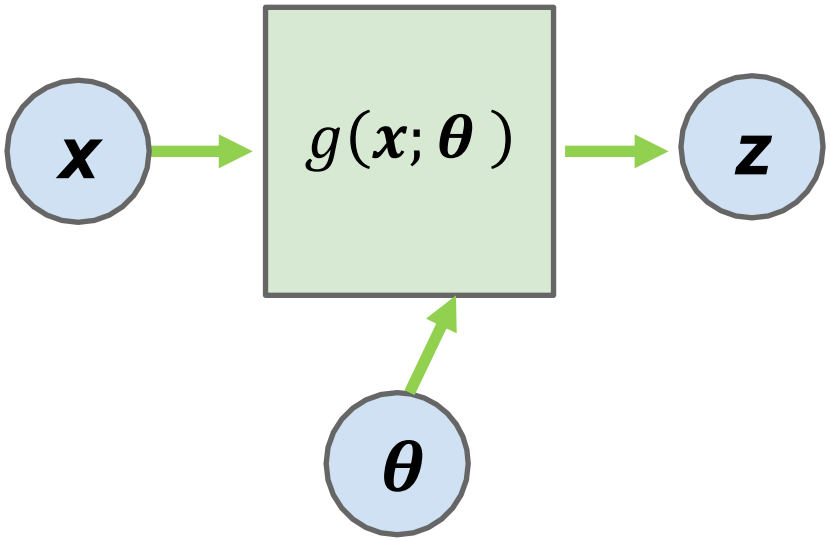
Backprop on Computational Graph





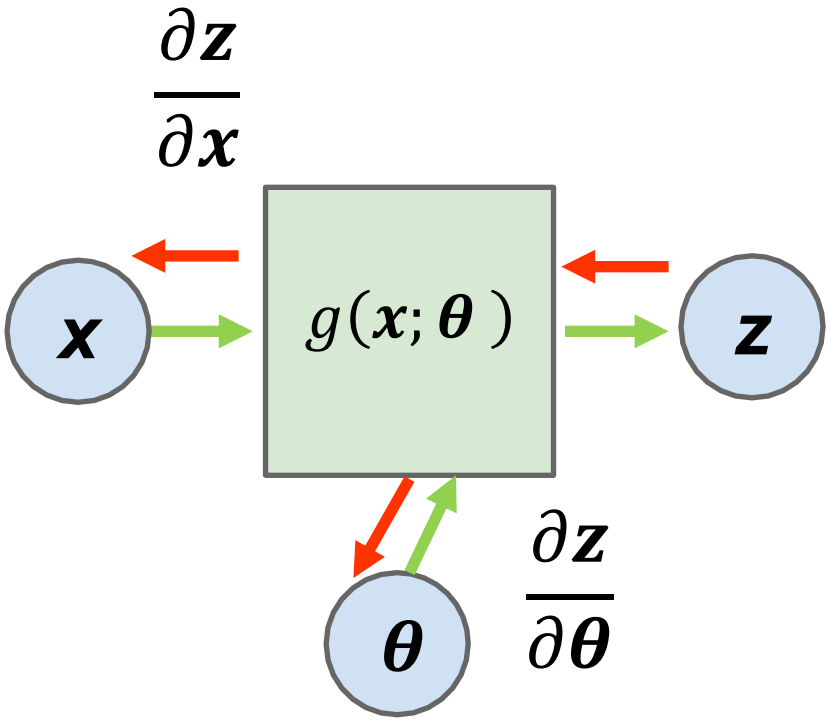
Computational Graph: Operators

- Differentiable operations
- Forward / backward



Computational Graph: Operators

- Differentiable operations
- Forward / backward



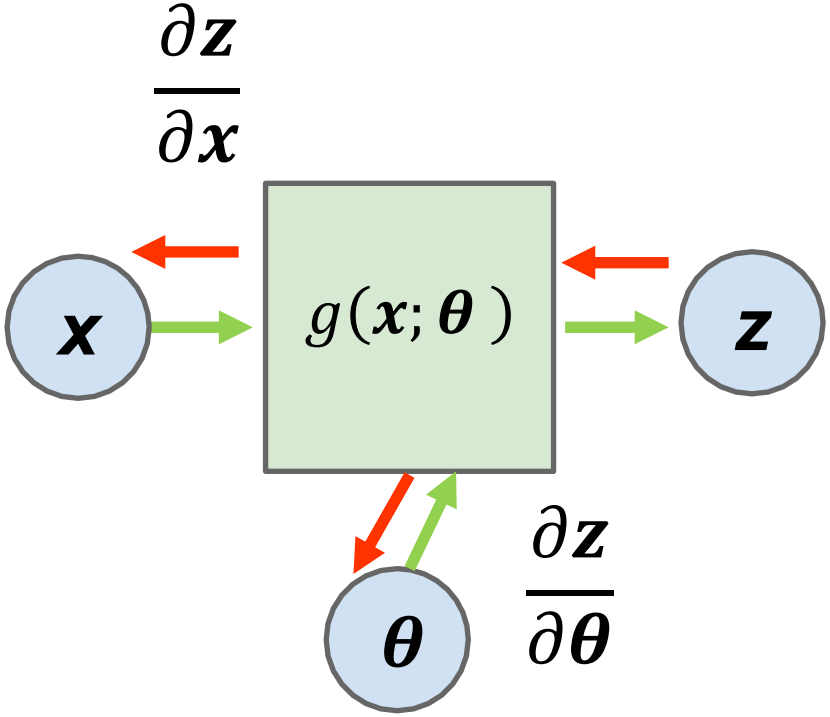
Computational Graph: Operators

- Differentiable operations
- Examples: addition

$$z = g(\boldsymbol{x}; \boldsymbol{b}) = \boldsymbol{x} + \boldsymbol{b}$$

$$\frac{\partial \boldsymbol{z}}{\partial \boldsymbol{x}} = \mathbf{1}$$

$$\frac{\partial \boldsymbol{z}}{\partial \boldsymbol{b}} = \mathbf{1}$$



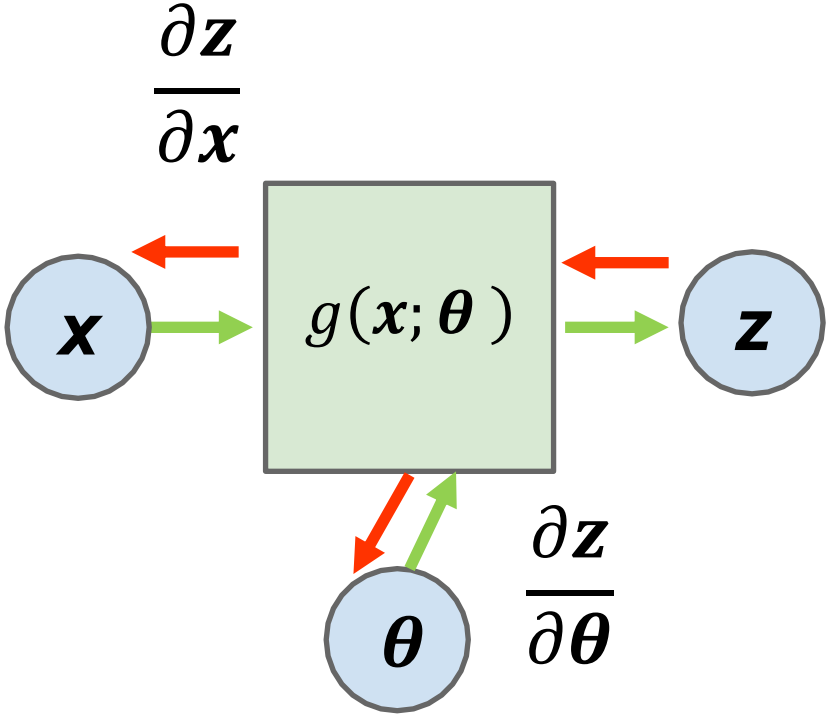
Computational Graph: Operators

- Differentiable operations
- Examples: vector-vector product

$$g(\boldsymbol{x}; \boldsymbol{\theta} = \boldsymbol{w}) = \boldsymbol{w}^T \boldsymbol{x}$$

$$\frac{\partial \boldsymbol{z}}{\partial \boldsymbol{x}} = diag(\boldsymbol{w})$$

$$\frac{\partial \boldsymbol{z}}{\partial \boldsymbol{w}} = diag(\boldsymbol{x})$$



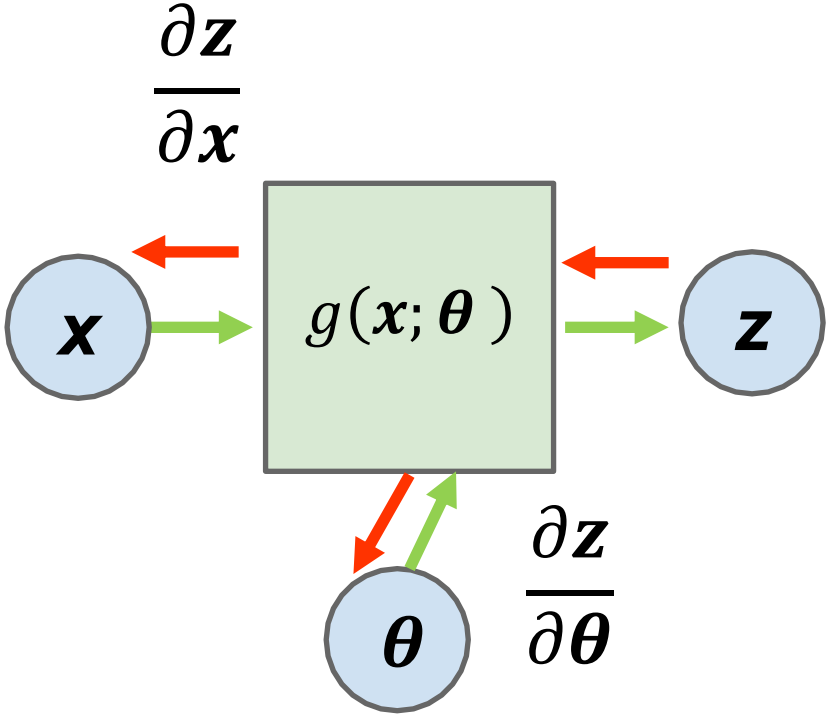
Computational Graph: Operators

- Differentiable operations
- Examples: matrix-vector product

$$g(\boldsymbol{x}; \boldsymbol{\theta} = \boldsymbol{W}) = \boldsymbol{W}\boldsymbol{x}$$

$$\frac{\partial \boldsymbol{z}}{\partial \boldsymbol{x}} = \boldsymbol{W}$$

$$\frac{\partial \boldsymbol{z}}{\partial \boldsymbol{W}} = \boldsymbol{I} \otimes \boldsymbol{x}^T$$

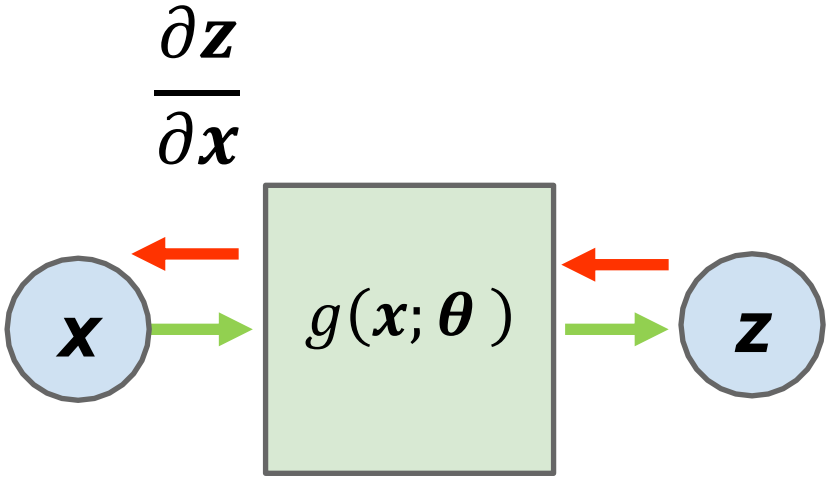


Computational Graph: Operators

- Differentiable operations
- Examples: ReLU

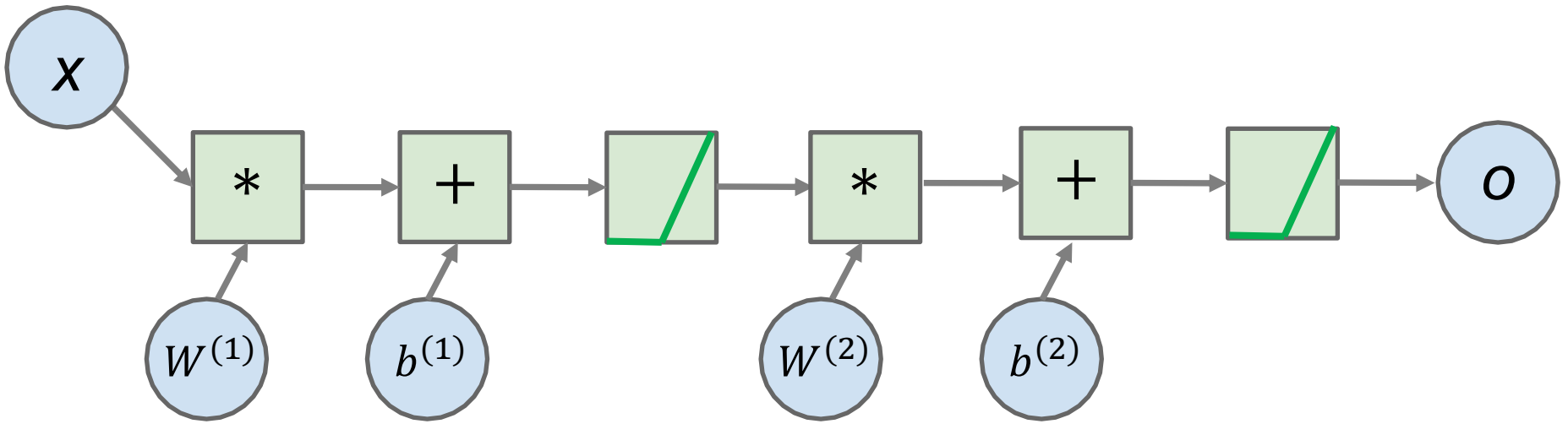
$$g(\boldsymbol{x}) = \max(0, x)$$

$$\frac{\partial \boldsymbol{z}}{\partial \boldsymbol{x}} = \begin{cases} 1 & x \geq 0 \\ 0 & x < 0 \end{cases}$$



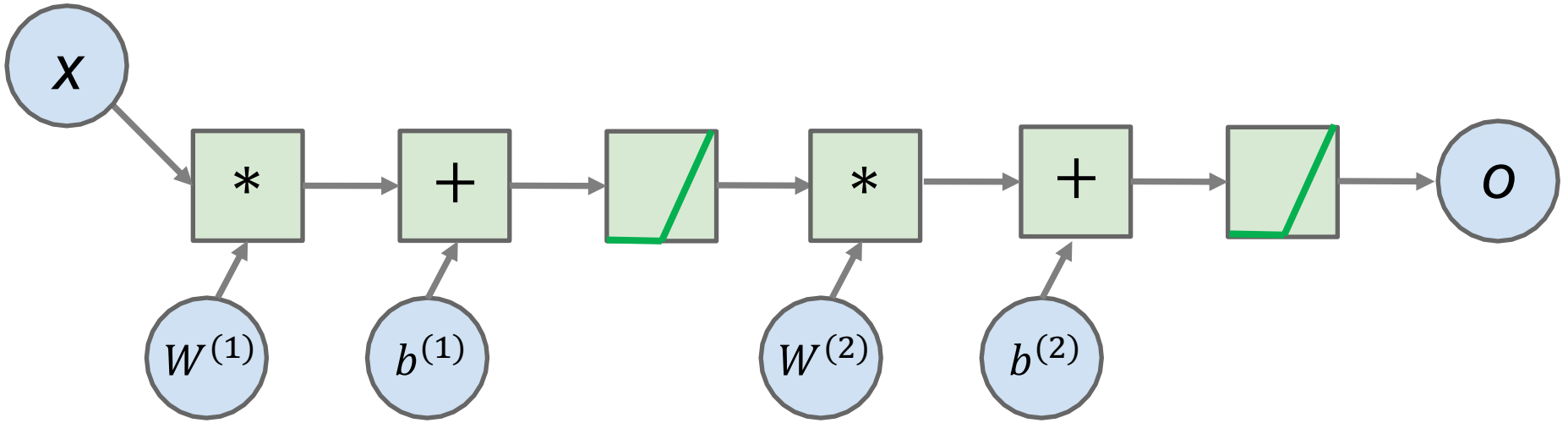
Neural network as a computational graph

- A two-layer neural network



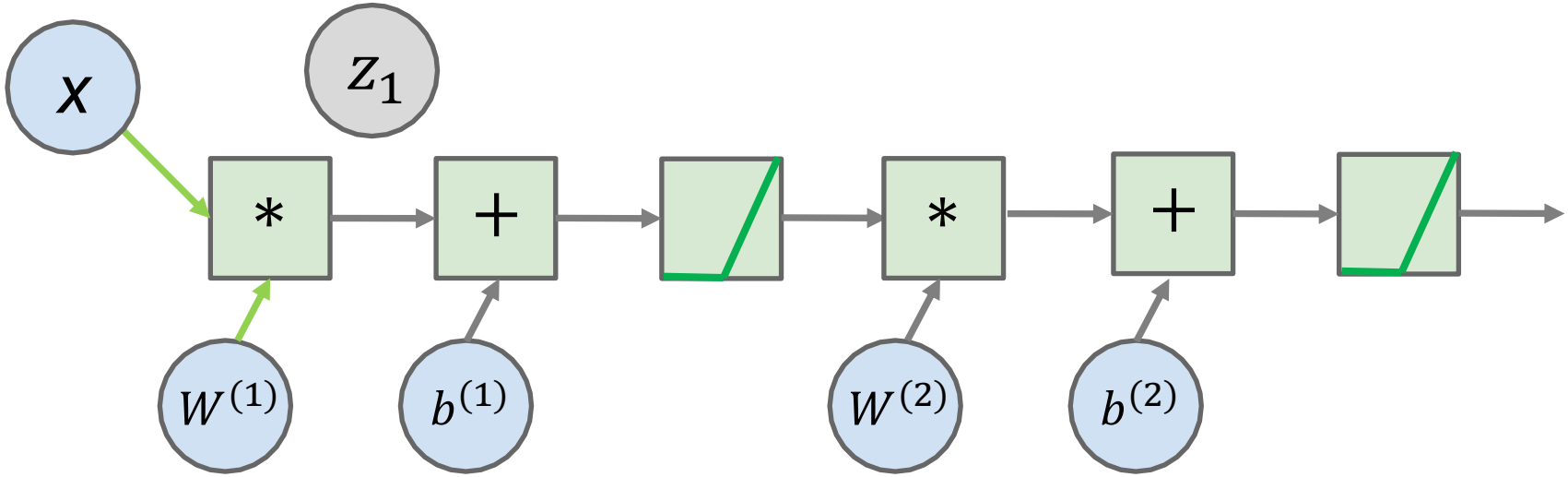
Neural network as a computational graph

- A two-layer neural network
- Forward propagation vs. backward propagation



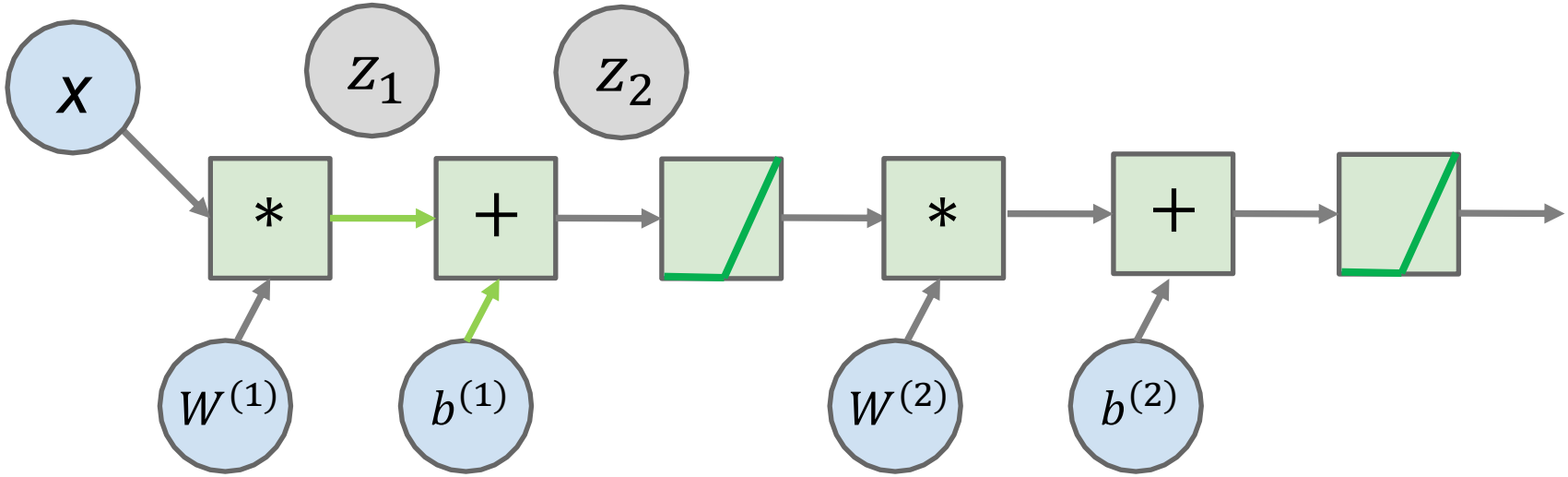
Neural network: forward propagation

- A two-layer neural network
- Intermediate variables Z



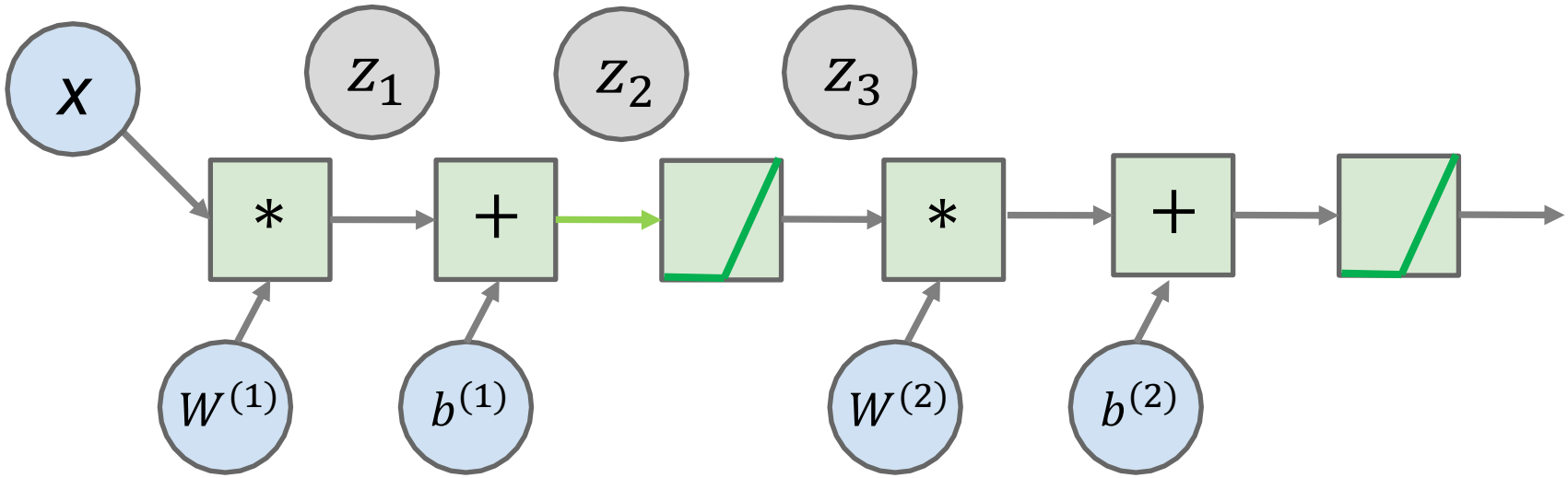
Neural network: forward propagation

- A two-layer neural network
- Intermediate variables Z



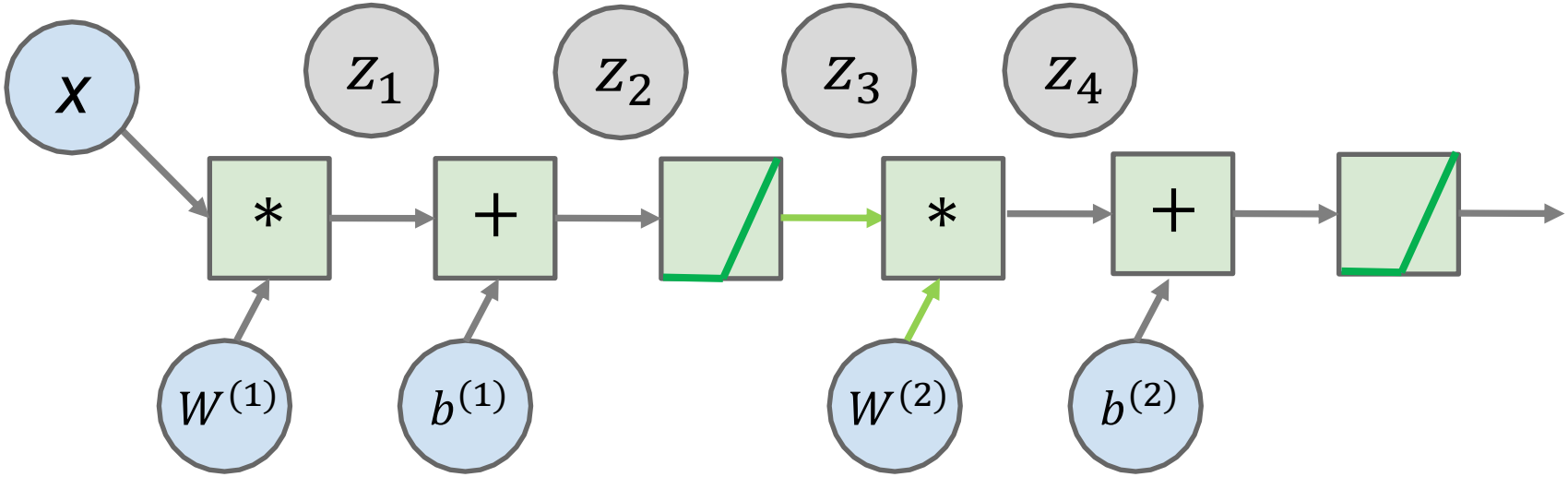
Neural network: forward propagation

- A two-layer neural network
- Intermediate variables Z



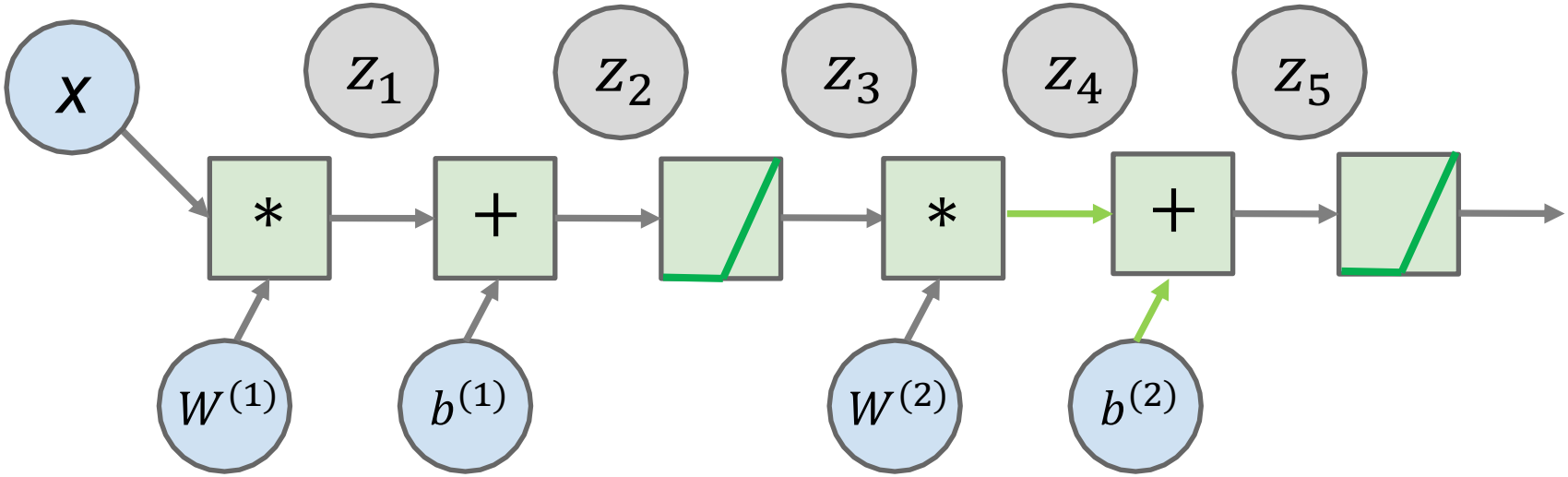
Neural network: forward propagation

- A two-layer neural network
- Intermediate variables Z



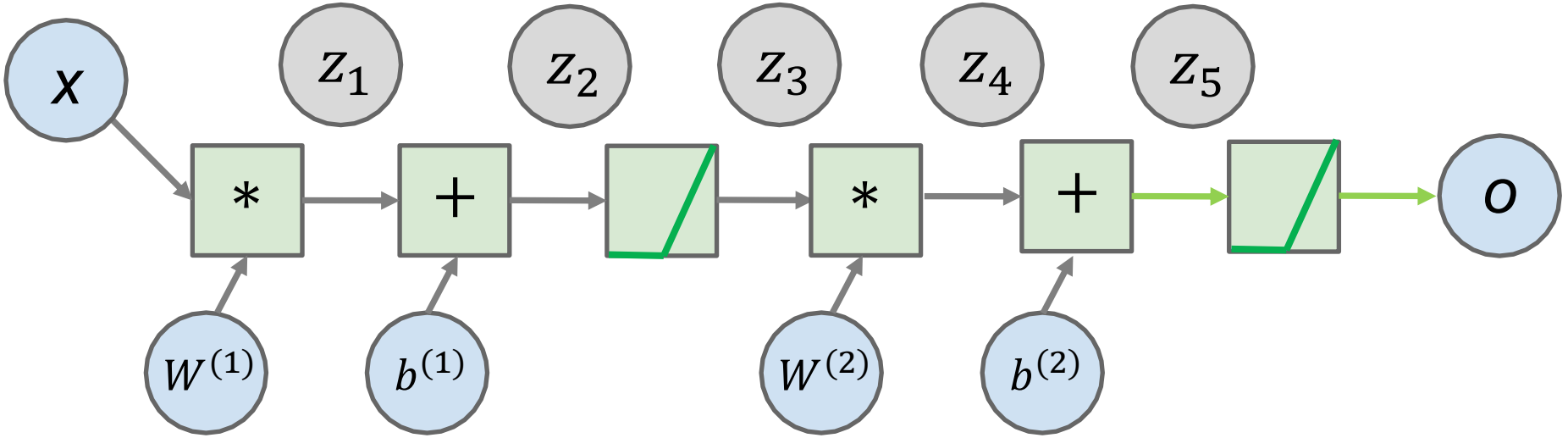
Neural network: forward propagation

- A two-layer neural network
- Intermediate variables *Z*



Neural network: forward propagation

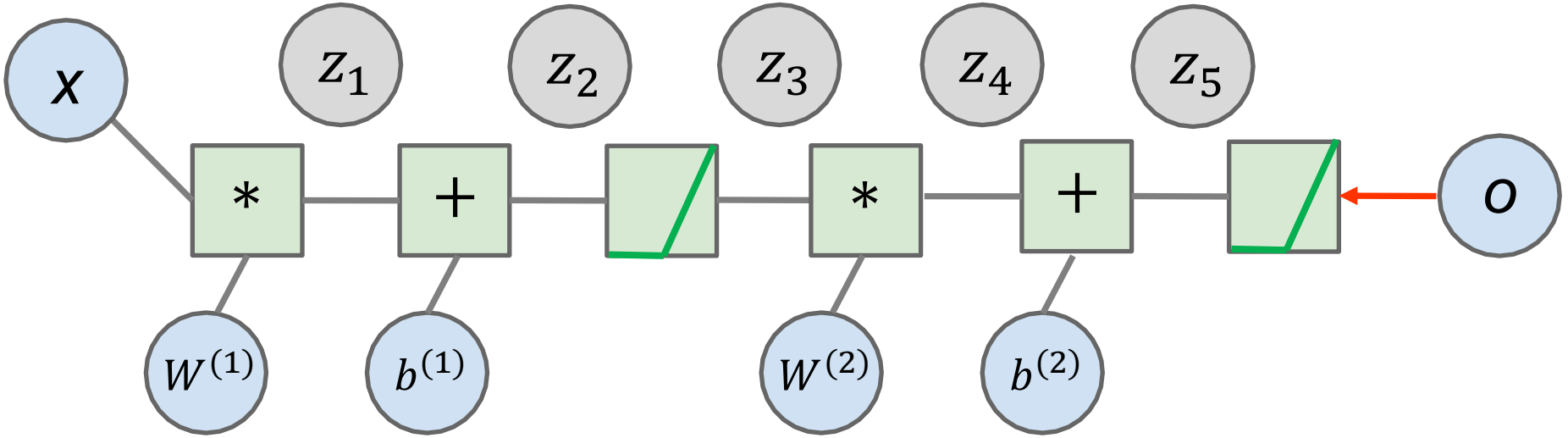
- A two-layer neural network
- Intermediate variables *Z*
- Forward propagation computes the output *o*



Neural network: backpropagation

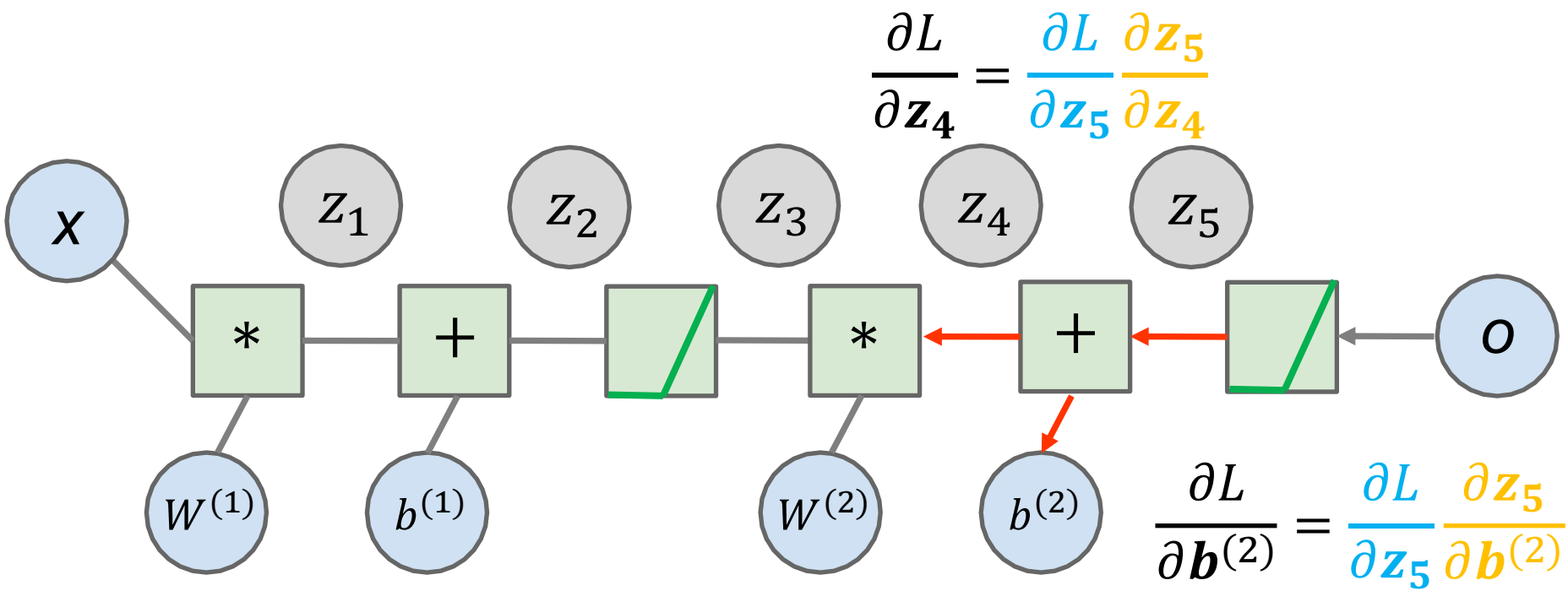
- A two-layer neural network
- Assuming forward propagation is done
- Have all intermediate variables Z
- Minimize a scalar loss function L

$$\frac{\partial L}{\partial \mathbf{z_5}} = \frac{\partial L}{\partial o}\frac{\partial \boldsymbol{o}}{\partial \boldsymbol{z_5}}$$



Neural network: backpropagation

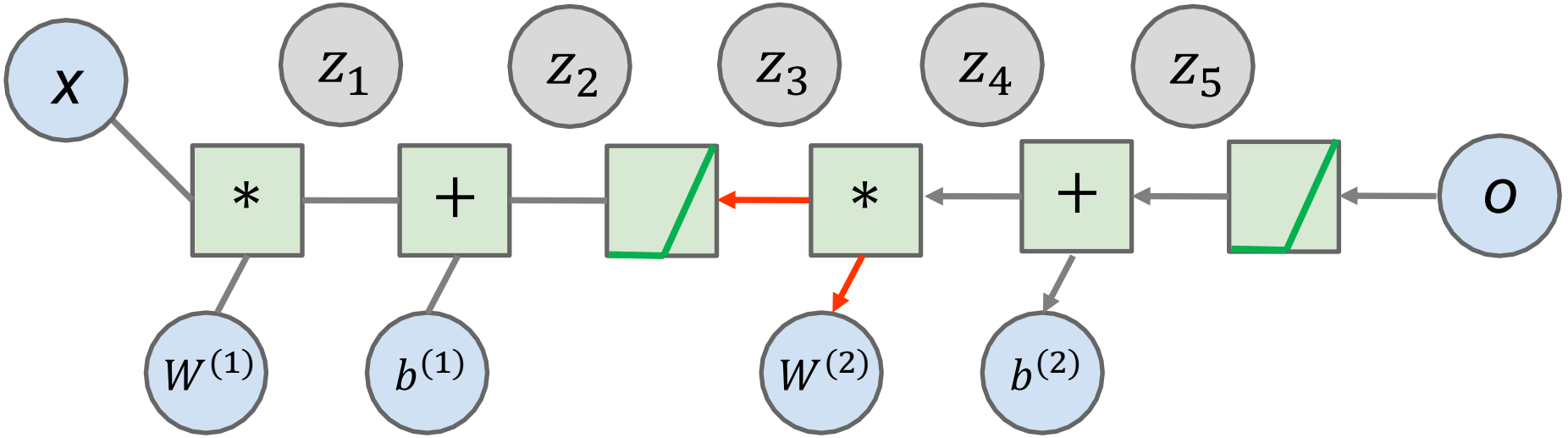
- A two-layer neural network
- Assuming forward propagation is done



Neural network: backpropagation

- A two-layer neural network
- Assuming forward propagation is done

$$\frac{\partial L}{\partial \boldsymbol{z_3}} = \frac{\partial L}{\partial \boldsymbol{z_4}} \frac{\partial \boldsymbol{z_4}}{\partial \boldsymbol{z_3}}$$



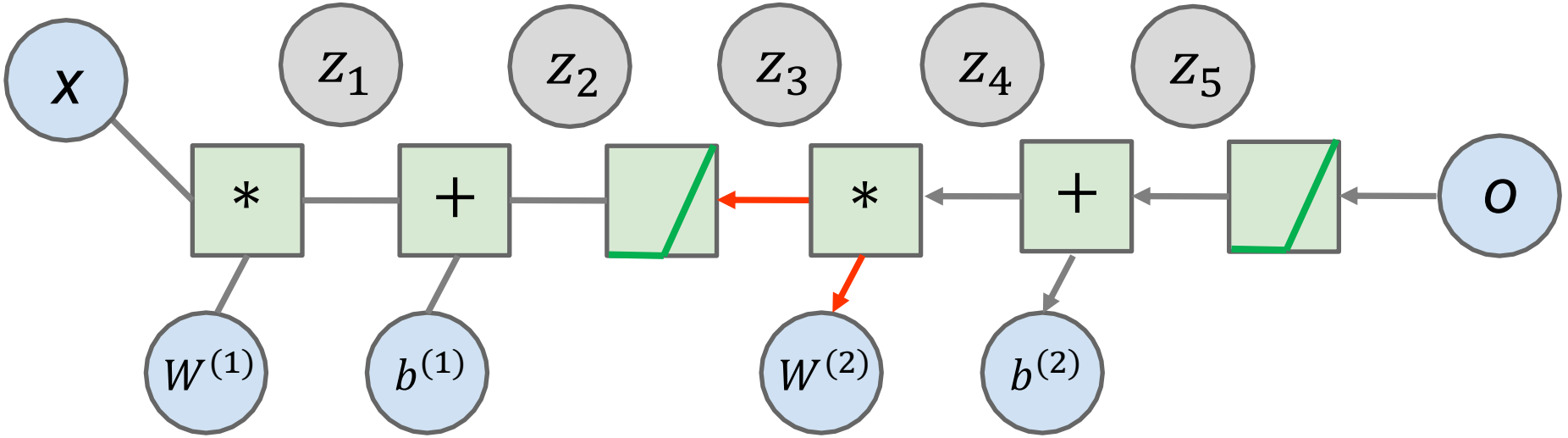
$$\frac{\partial L}{\partial \boldsymbol{W}^{(2)}} = \frac{\partial L}{\partial \boldsymbol{z_4}} \frac{\partial \boldsymbol{z_4}}{\partial \boldsymbol{W}^{(2)}}$$

Neural network: backpropagation

- Gradient to a variable =

gradient on the top x gradient from the current operation

$$\frac{\partial L}{\partial \boldsymbol{z_3}} = \frac{\partial L}{\partial \boldsymbol{z_4}} \frac{\partial \boldsymbol{z_4}}{\partial \boldsymbol{z_3}}$$



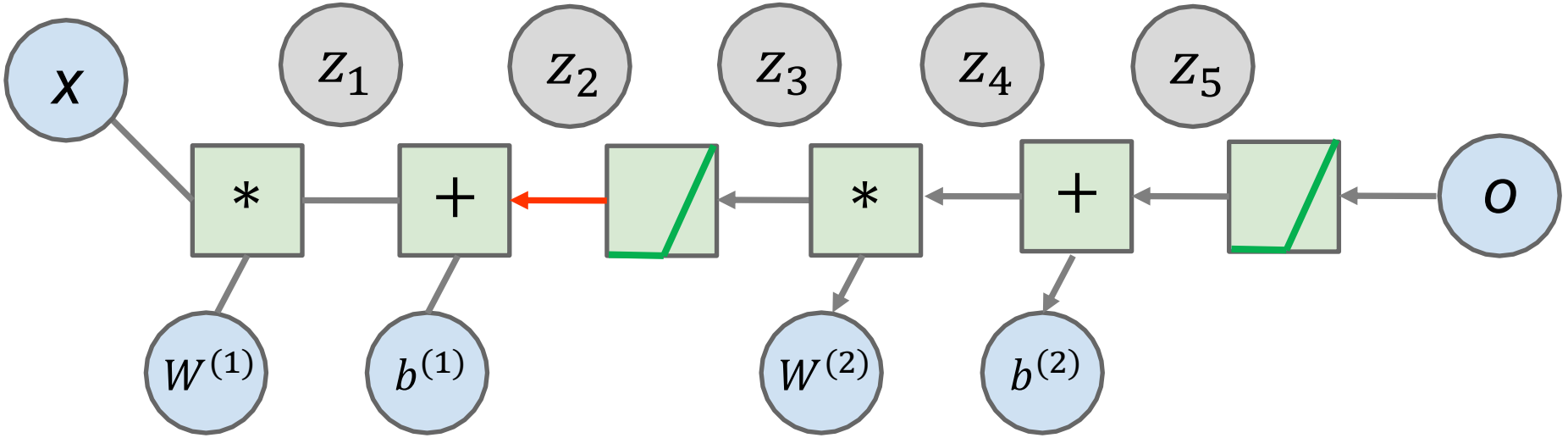
$$\frac{\partial L}{\partial \boldsymbol{W}^{(2)}} = \frac{\partial L}{\partial \boldsymbol{z_4}} \frac{\partial \boldsymbol{z_4}}{\partial \boldsymbol{W}^{(2)}}$$

Neural network: backpropagation

- Gradient to a variable =

gradient on the top x gradient from the current operation

$$\frac{\partial L}{\partial \mathbf{z_2}} = \frac{\partial L}{\partial \mathbf{z_3}} \frac{\partial \mathbf{z_3}}{\partial \mathbf{z_2}}$$



Reverse mode automatic differentiation

(size of gradient = size of parameters tensor)

Backpropagation: a modern treatment

- Define a neural network as a computational graph
  - Must be a *directed* graph
  - Nodes as *variables* and *operations*
  - All operations must be differentiable

Backpropagation: a modern treatment

- Define a neural network as a computational graph
- Run topological sort to determine the order of execution of the operations (input -> output)
- Forward propagation to compute the output
  - Also store all intermediate outputs
- Compare the output with target label (the loss function)
- Backward propagation to compute the gradients w.r.t. the model parameters
  - gradient on the top x gradient from the current op
  - size of gradients = size of parameters
- Use gradient descent to update the parameters

Summary

- Visual Recognition as (supervised) learning
- An example of fully connected neural networks
- Basics of neural networks (forward / backward propagation, training)
- Neural networks as computational graphs

To Be Continued